



# NLP项目

## Attention

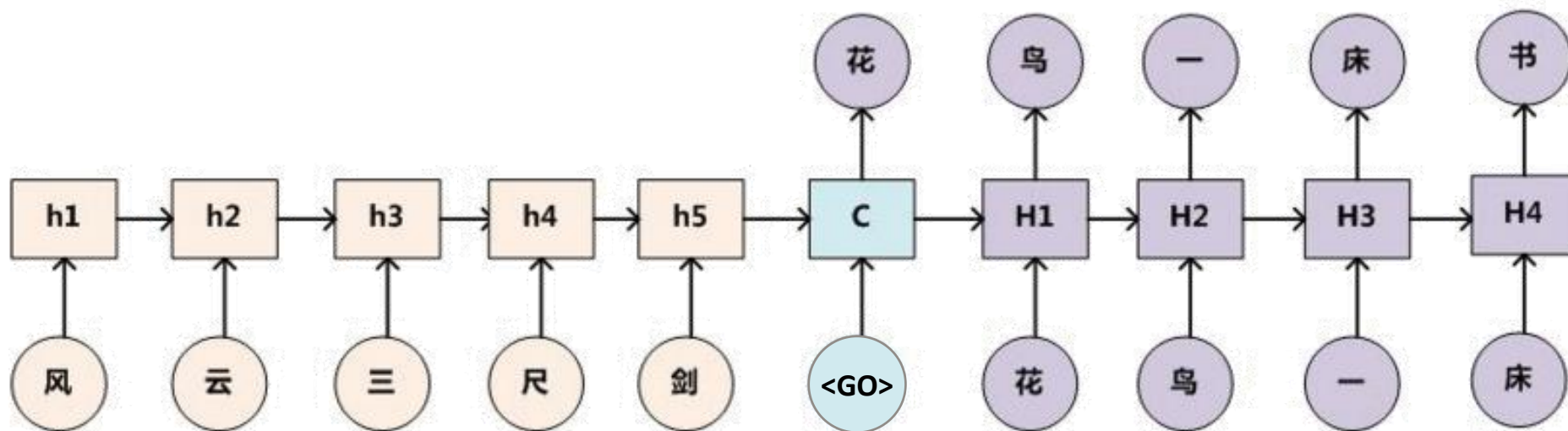
# 课程内容

🕒 **Attention 结构讲解**

🕒 **Seq2Seq+Attention**

# Seq2Seq问题

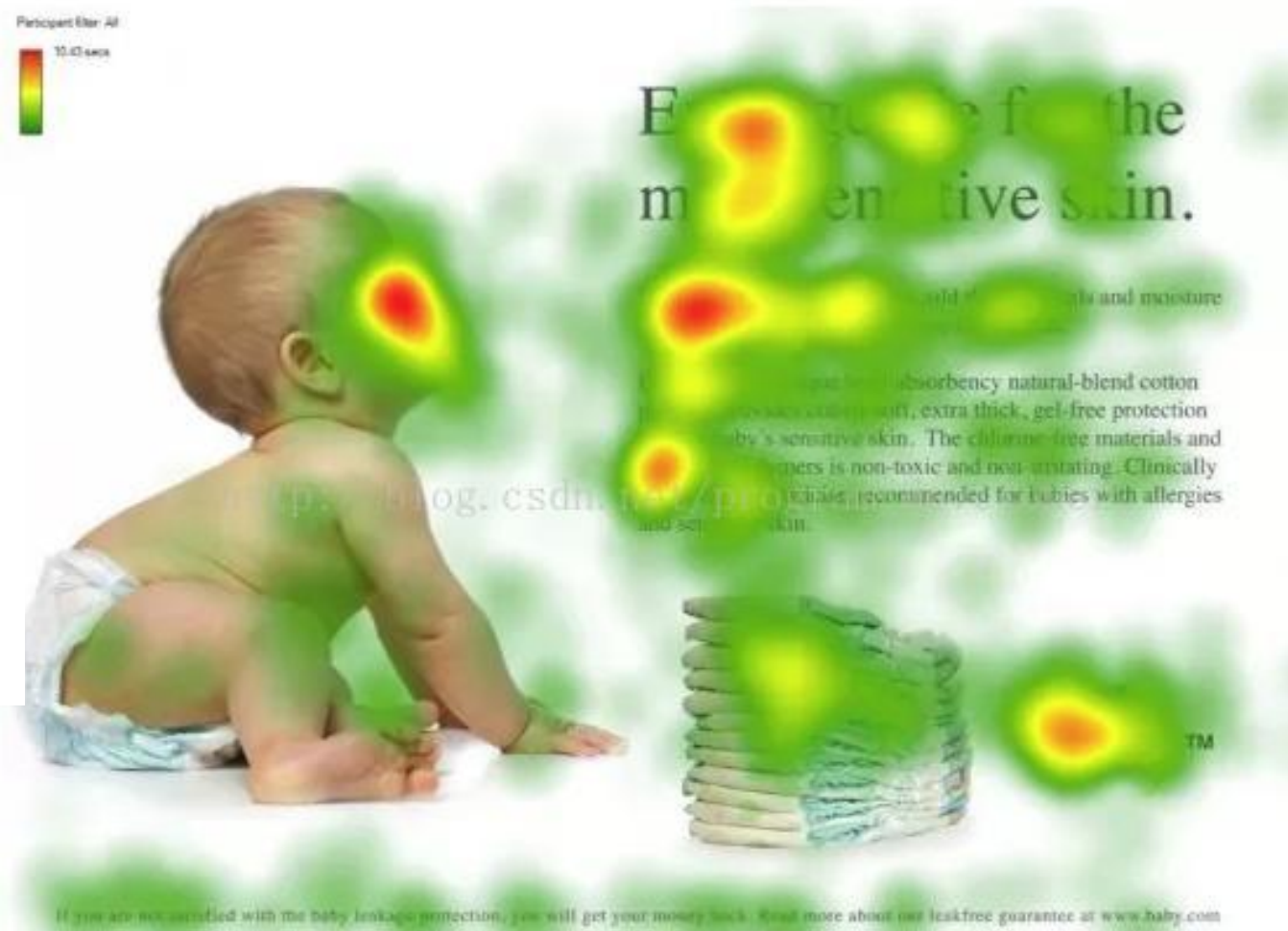
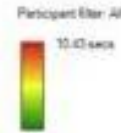
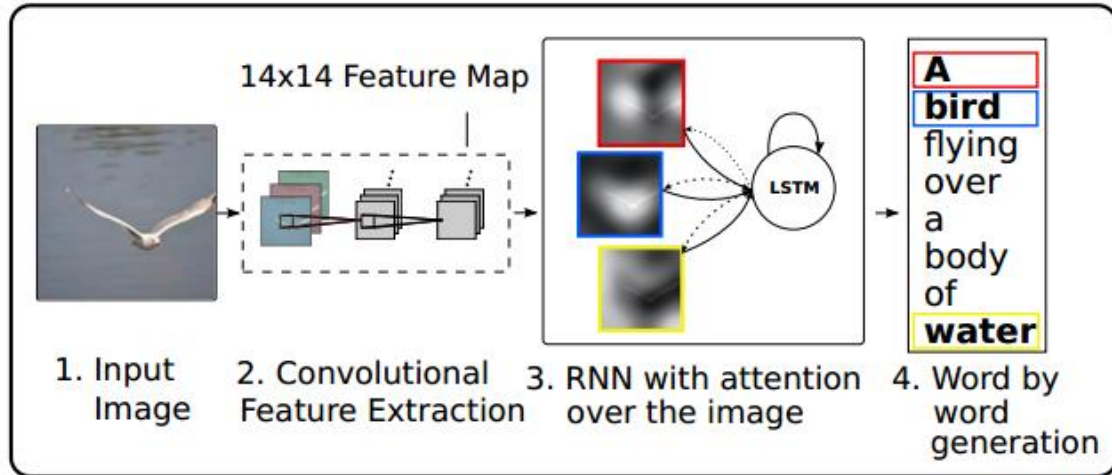
⌚ 问题描述：“风”对应的特征对于下联的影响是最弱的。



下联生成的时候，不仅仅需要考虑上联的整个语义文本信息，针对下联里面的每个token(字)都需要和上联的对应位置的token(字)进行“关联”

1. 字句对等；
2. 词性对品；
3. 结构对应；
4. 节律对拍；
5. 平仄对立；
6. 形对意联；

# Attention



If you are not satisfied with the baby leakage protection, you will get your money back. Read more about our leakfree guarantee at [www.baby.com](http://www.baby.com)

# Seq2Seq Attention

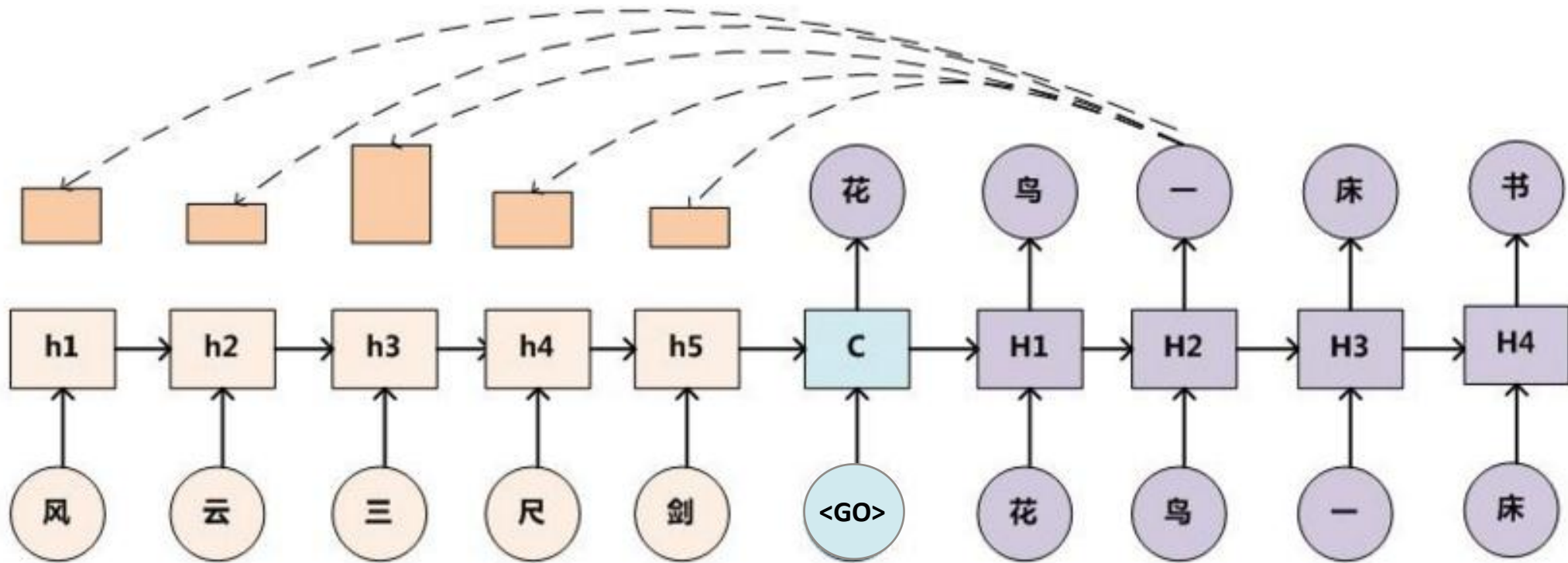
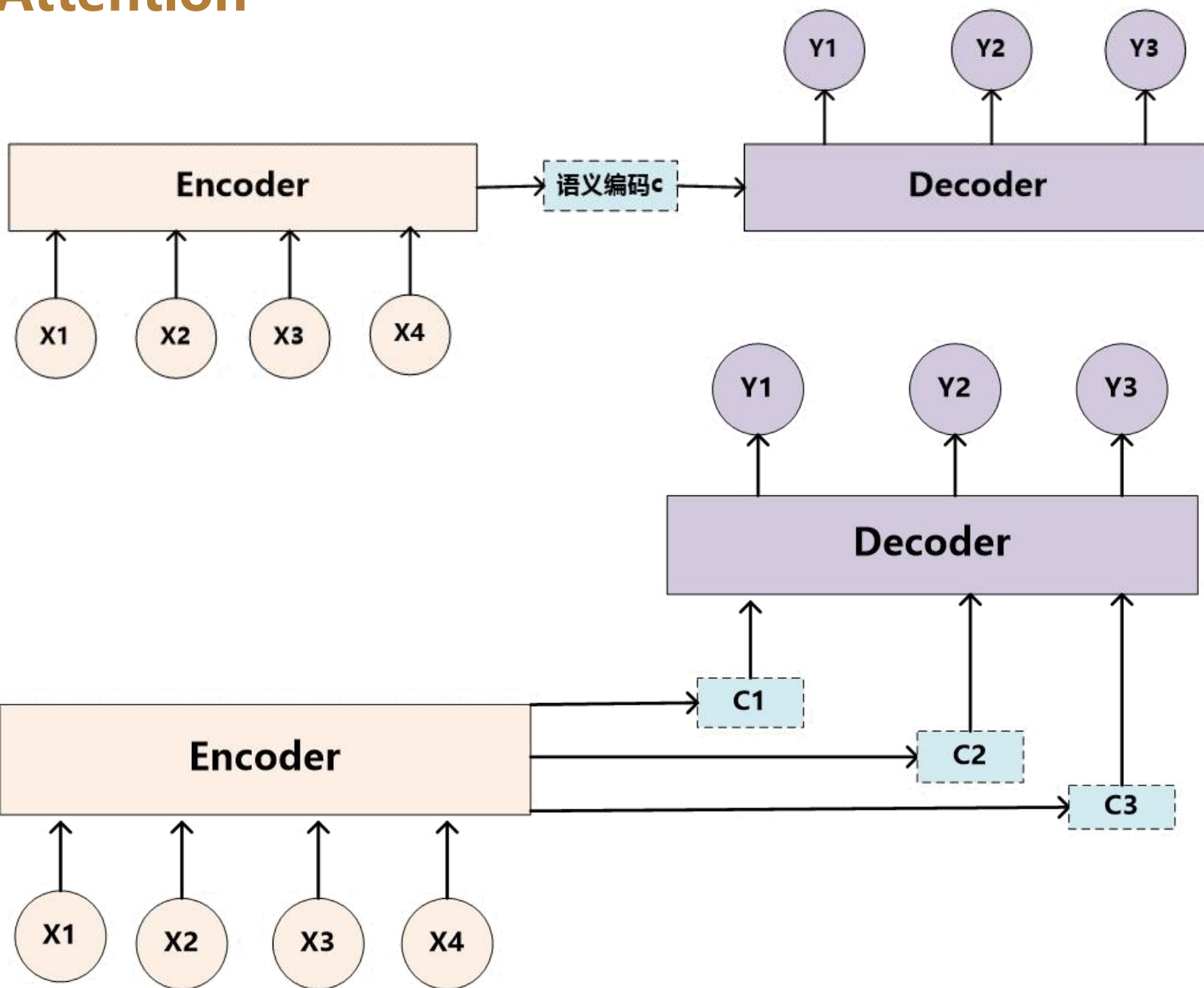


图3. Attention模型



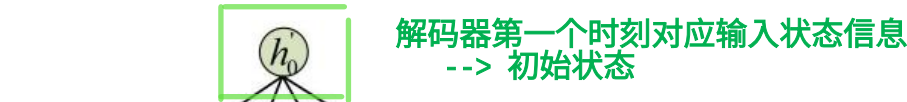
# Seq2Seq Attention



# Attention对齐机制(词对齐)

我                  爱                  中                  国

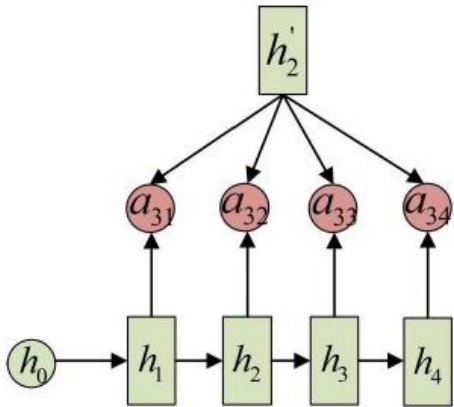
$$\begin{bmatrix} h_1 \end{bmatrix} * \begin{bmatrix} a_{11} \end{bmatrix} + \begin{bmatrix} h_2 \end{bmatrix} * \begin{bmatrix} a_{12} \end{bmatrix} + \begin{bmatrix} h_3 \end{bmatrix} * \begin{bmatrix} a_{13} \end{bmatrix} + \begin{bmatrix} h_4 \end{bmatrix} * \begin{bmatrix} a_{14} \end{bmatrix} = \begin{bmatrix} c_1 \end{bmatrix} \longrightarrow \text{I}$$
$$\begin{bmatrix} h_1 \end{bmatrix} * \begin{bmatrix} a_{21} \end{bmatrix} + \begin{bmatrix} h_2 \end{bmatrix} * \begin{bmatrix} a_{22} \end{bmatrix} + \begin{bmatrix} h_3 \end{bmatrix} * \begin{bmatrix} a_{23} \end{bmatrix} + \begin{bmatrix} h_4 \end{bmatrix} * \begin{bmatrix} a_{24} \end{bmatrix} = \begin{bmatrix} c_2 \end{bmatrix} \longrightarrow \text{Love}$$
$$\begin{bmatrix} h_1 \end{bmatrix} * \begin{bmatrix} a_{31} \end{bmatrix} + \begin{bmatrix} h_2 \end{bmatrix} * \begin{bmatrix} a_{32} \end{bmatrix} + \begin{bmatrix} h_3 \end{bmatrix} * \begin{bmatrix} a_{33} \end{bmatrix} + \begin{bmatrix} h_4 \end{bmatrix} * \begin{bmatrix} a_{34} \end{bmatrix} = \begin{bmatrix} c_3 \end{bmatrix} \longrightarrow \text{China}$$



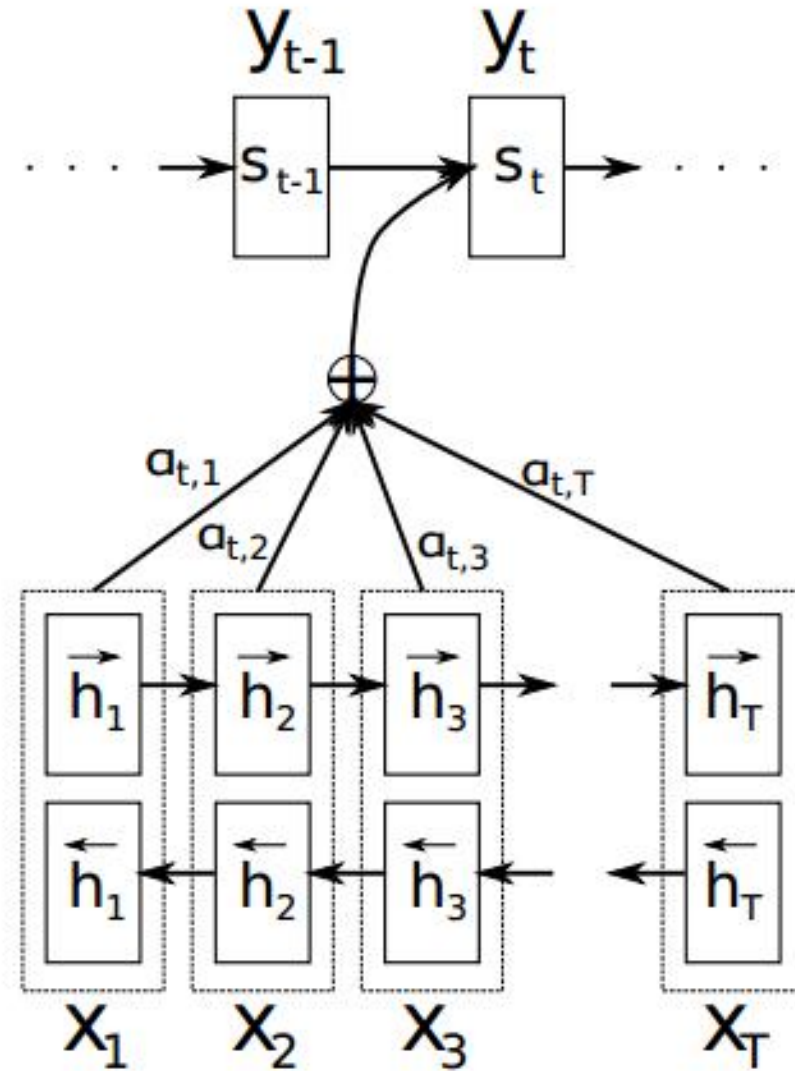
解码器第一个时刻对应输入状态信息  
--> 初始状态



解码器第一个时刻输出的状态信息 --> 第二个时刻对应输入状态信息

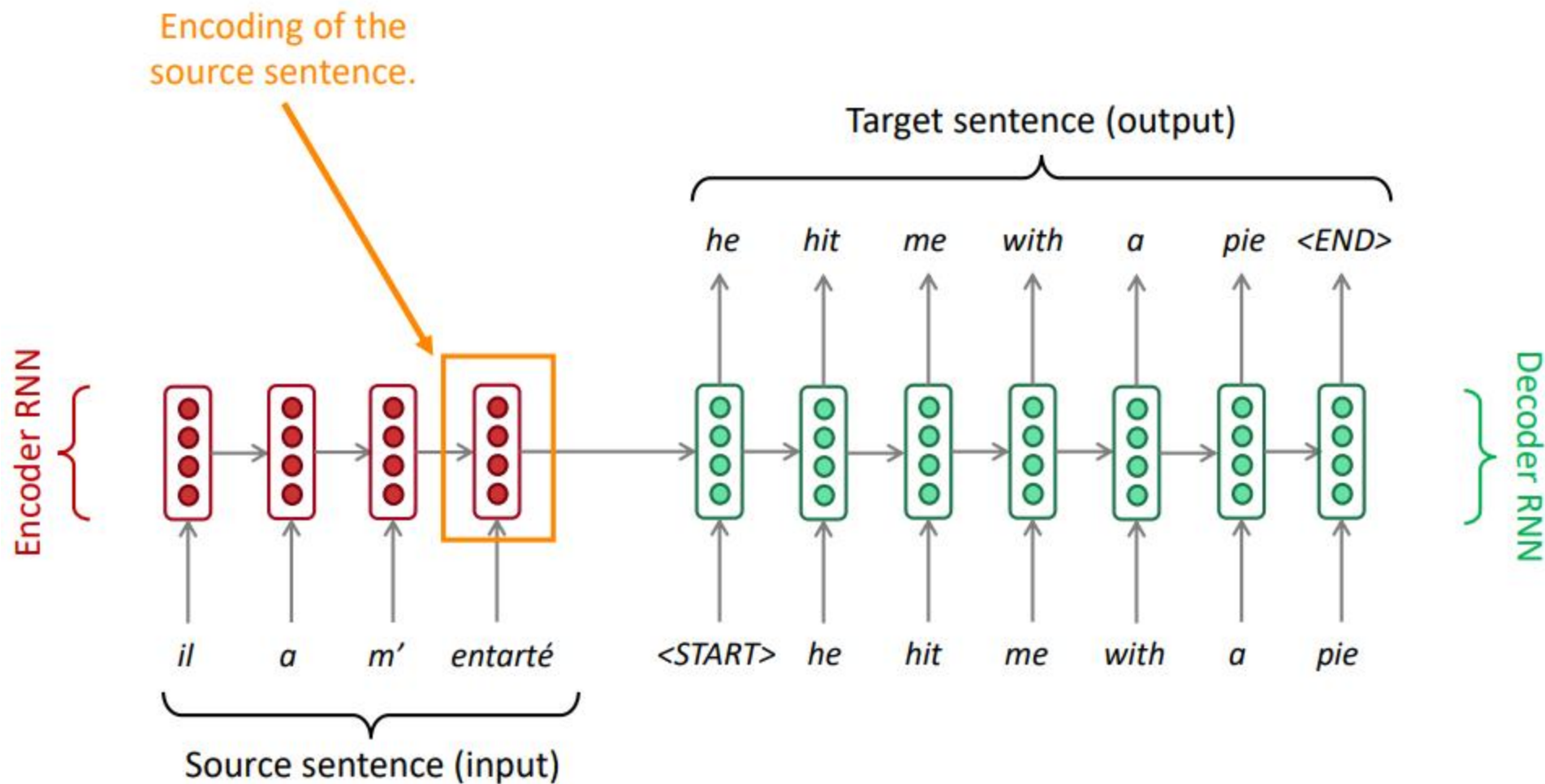


# Seq2Seq Attention



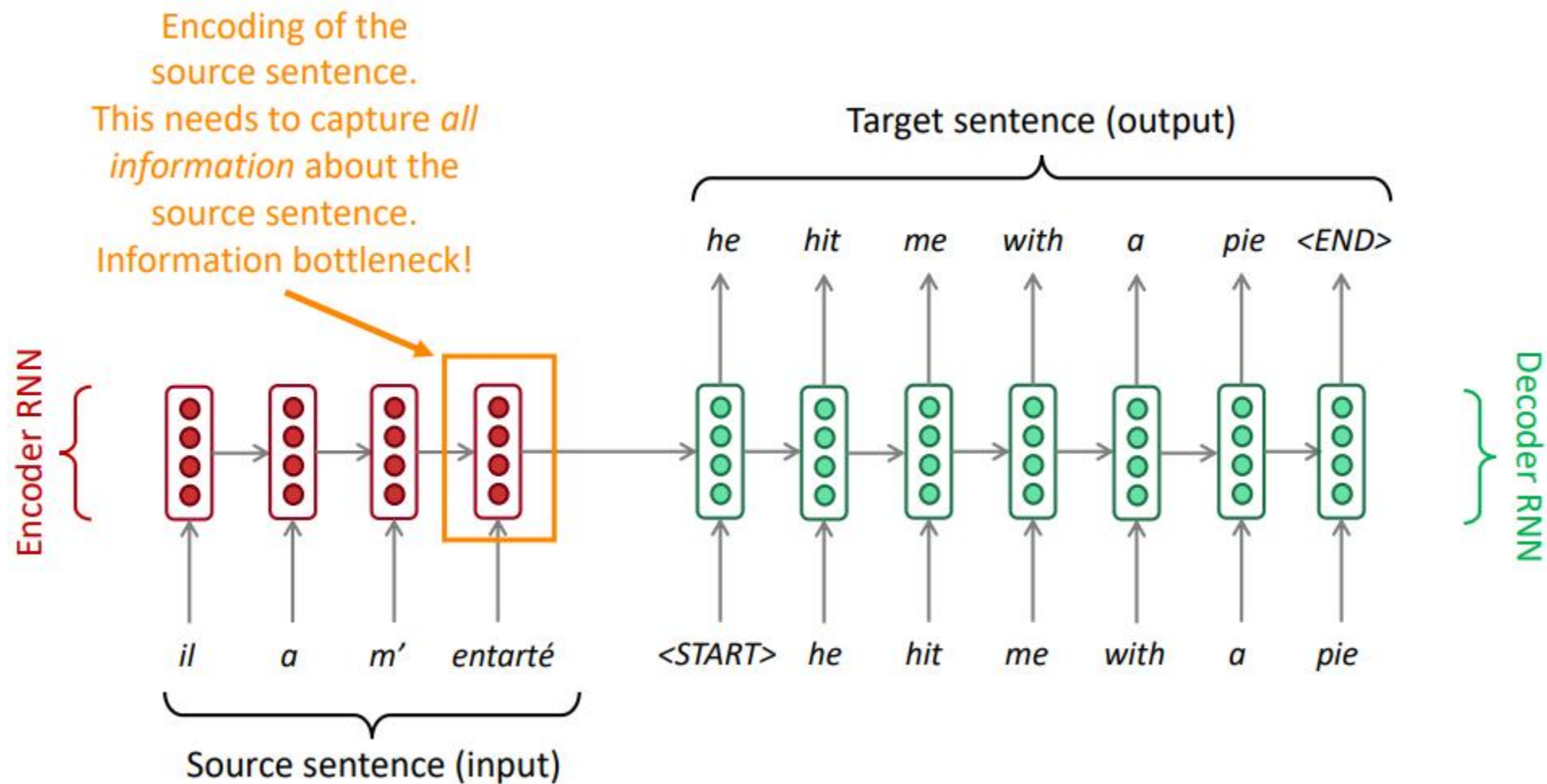


# Seq2Seq Attention计算过程

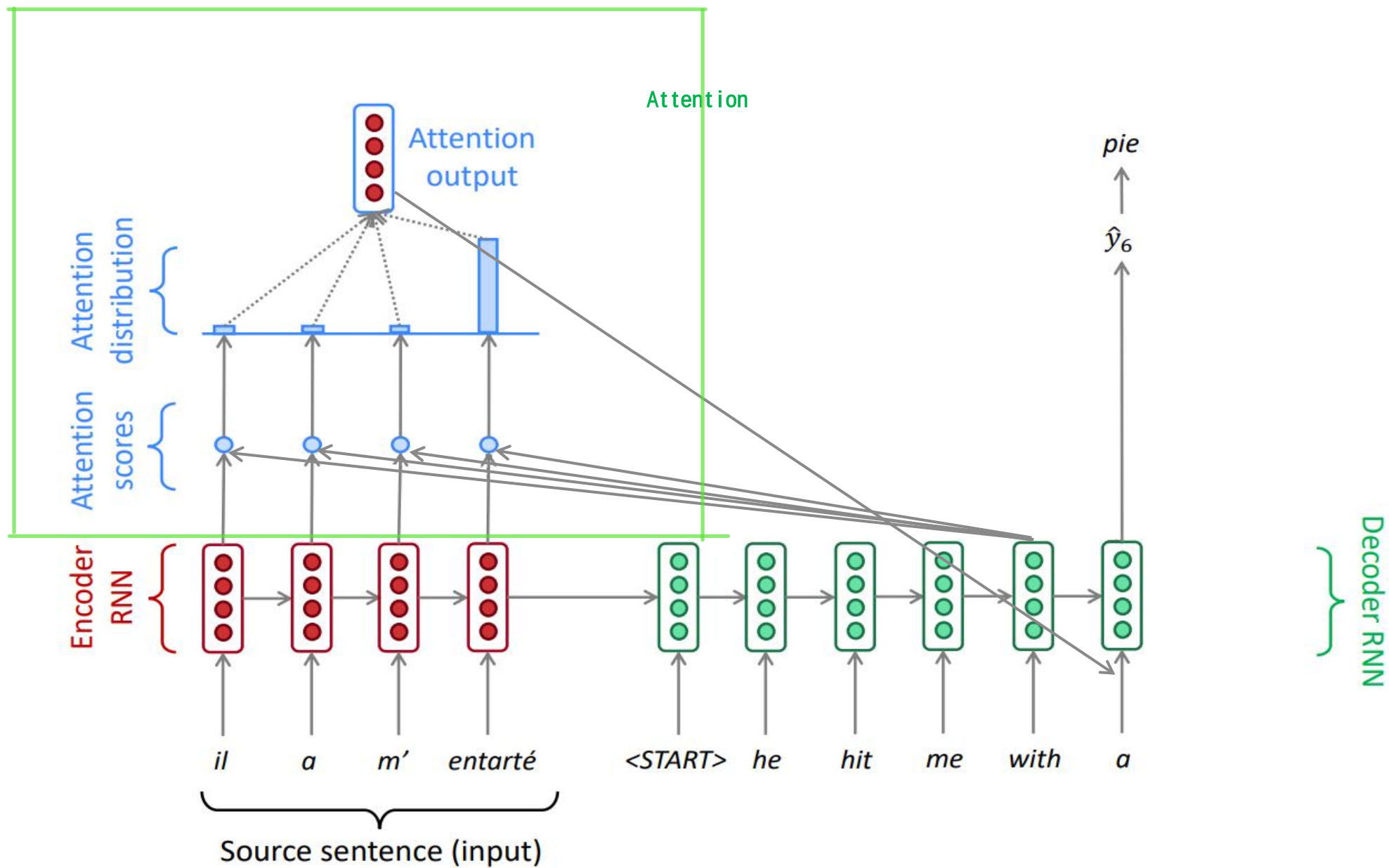


Problems with this architecture?

# Seq2Seq Attention计算过程



# Seq2Seq Attention计算过程



## Seq2Seq Attention计算过程

⌚ Encoder hidden states/output values:  $h_i$ ; 总Encoder时刻n个。

⌚ 时刻t, Decoder hidden states:  $s_t$

⌚ 基于每个时刻Encoder输出以及上一个时刻Decoder的状态来构建Attention

Scores:  $e_{t,i} = F(h_i, s_{t-1})$   $e_t = (e_{t,1}, e_{t,2}, \dots, e_{t,n})$  Attention

⌚ 对e进行softmax转换, 得到概率分布:  $\alpha_t = \text{softmax}(e_t)$

⌚ 基于概率分布以及所有Encoder的输出计算出Attention值;  $a_t = \sum_{i=1}^n \alpha_{t,i} h_i$

⌚ 将Decoder当前时刻的输入和Attention值结合形成新的输入数据, 然后进行普通的RNN Decoder操作。

$$y'_t = [y_t; a_t]$$

# Seq2Seq Attention计算过程

⌚ Attention Scores的计算函数F在不同论文中有很多形式，主要方式如下：

⌚ 乘法Attention:  $e_{t,i} = s_{t-1}^T h_i$

$$e_{t,i} = s_{t-1}^T h_i / \sqrt{d}$$

现有的框架中应用最多的方式

⌚ 加法Attention:  $e_{t,i} = s_{t-1}^T W h_i$

$$e_{t,i} = u^T \tanh(W_1 h_i + W_2 s_{t-1})$$

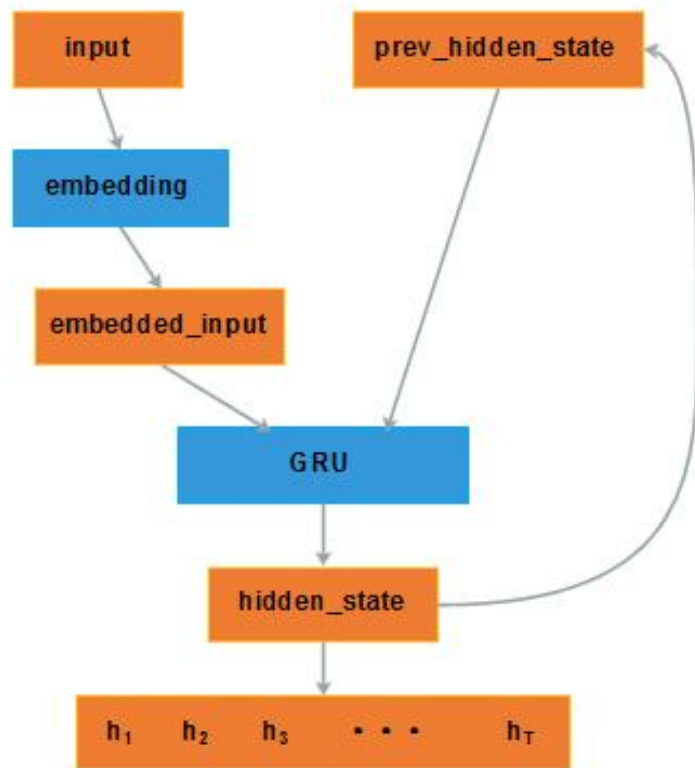
$$e_{t,i} = W_1 h_i + W_2 s_{t-1}$$

$$e_{t,i} = W h_i$$

TensorFlow默认

# Seq2Seq Attention计算过程

## Bi-RNN Encoder



Encoder的流程如上图所示，最终的输出结果是每个时刻的hidden\_state  $h_1, h_2, h_3, \dots, h_T$ 。

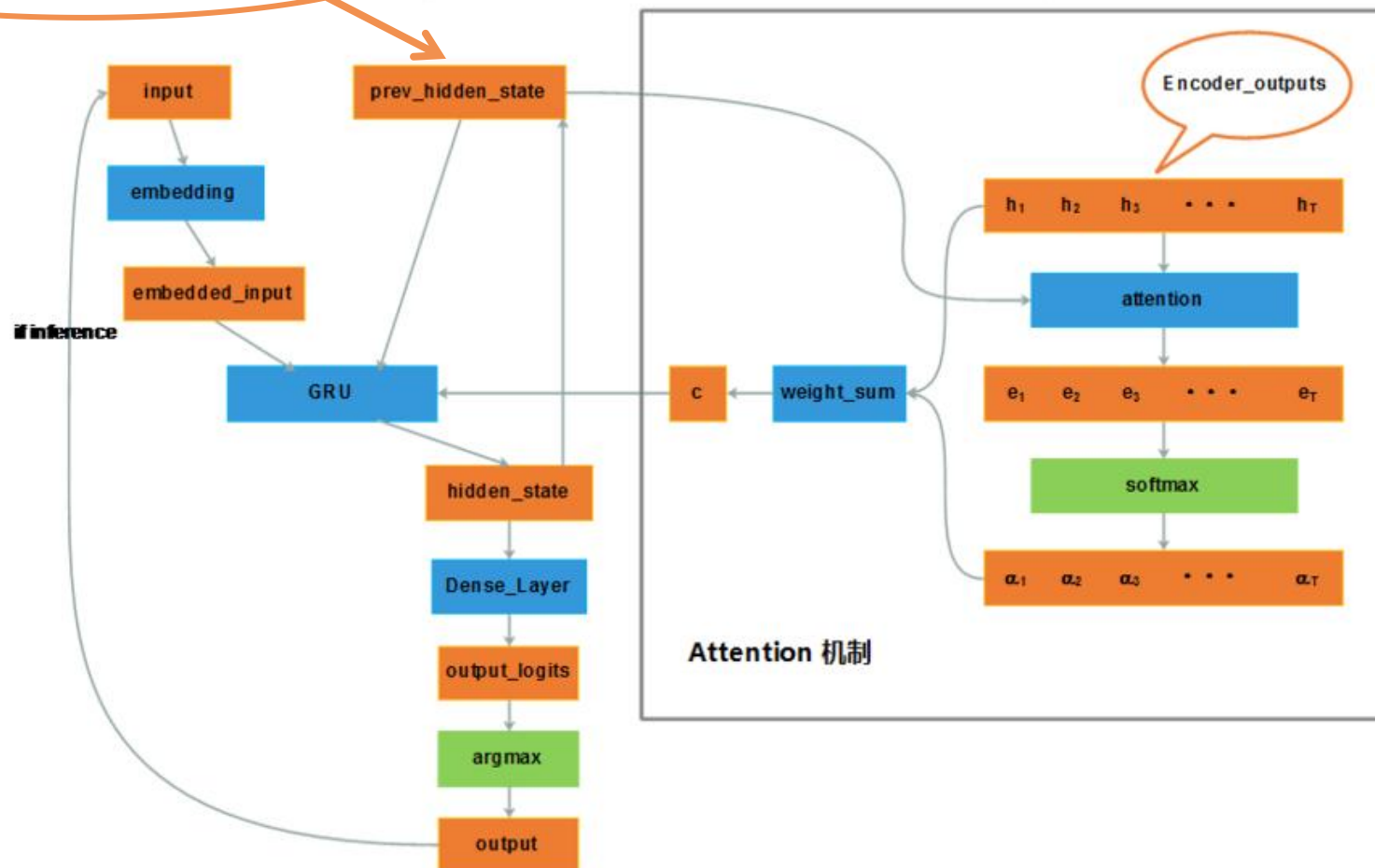


# Seq2Seq Attention计算过程

## Attention-Decoder

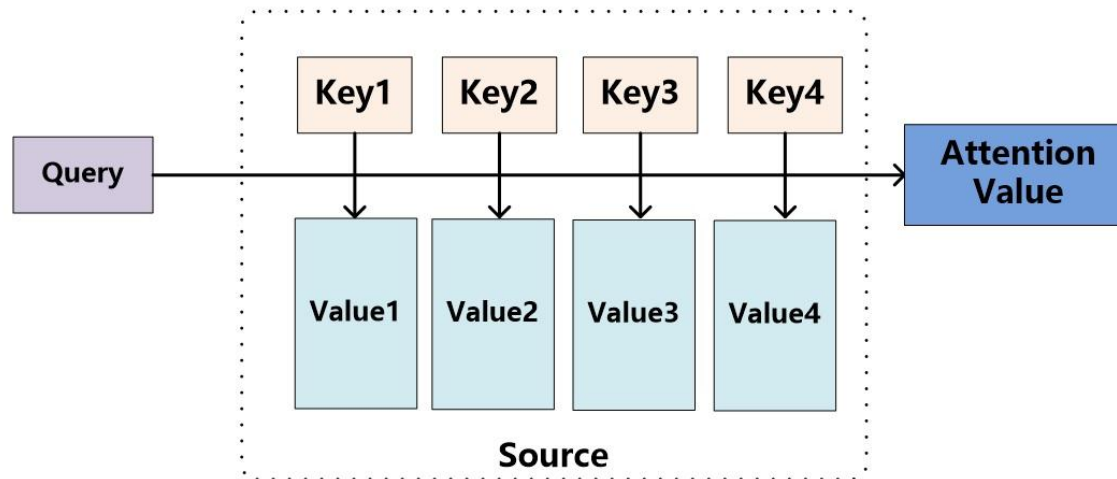
Initial state by Encoder States/Outputs( $h_T$ )

## Attention-Decoder for each time step



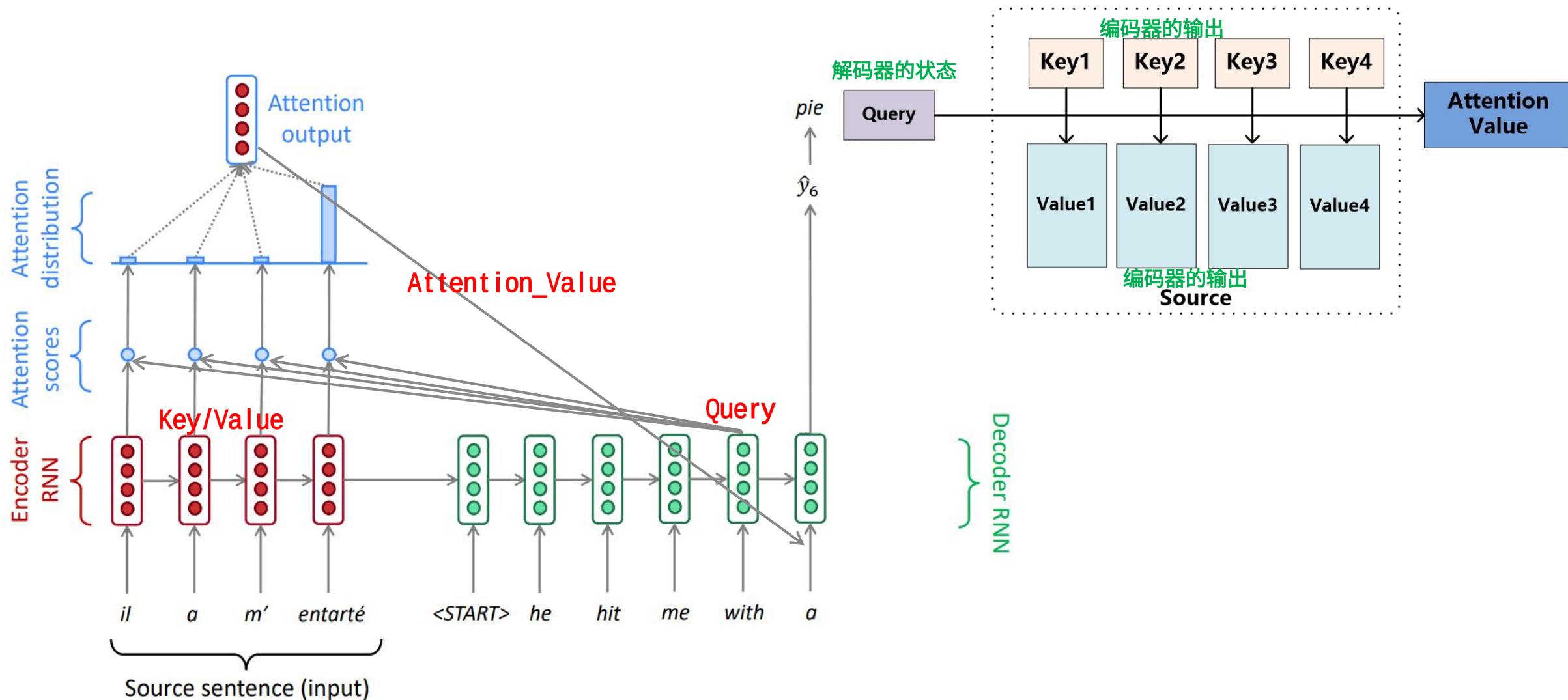
## Seq2Seq Attention计算过程(另一种理解方式)

- ⌚ 此时给定Target中的某个元素Query，通过计算Query和各个Key的相似性或者相关性，得到每个Key对应Value的权重系数，然后对Value进行加权求和，即得到了最终的Attention数值。所以**本质上Attention机制是对Source中元素的Value值进行加权求和，而Query和Key用来计算对应Value的权重系数。**

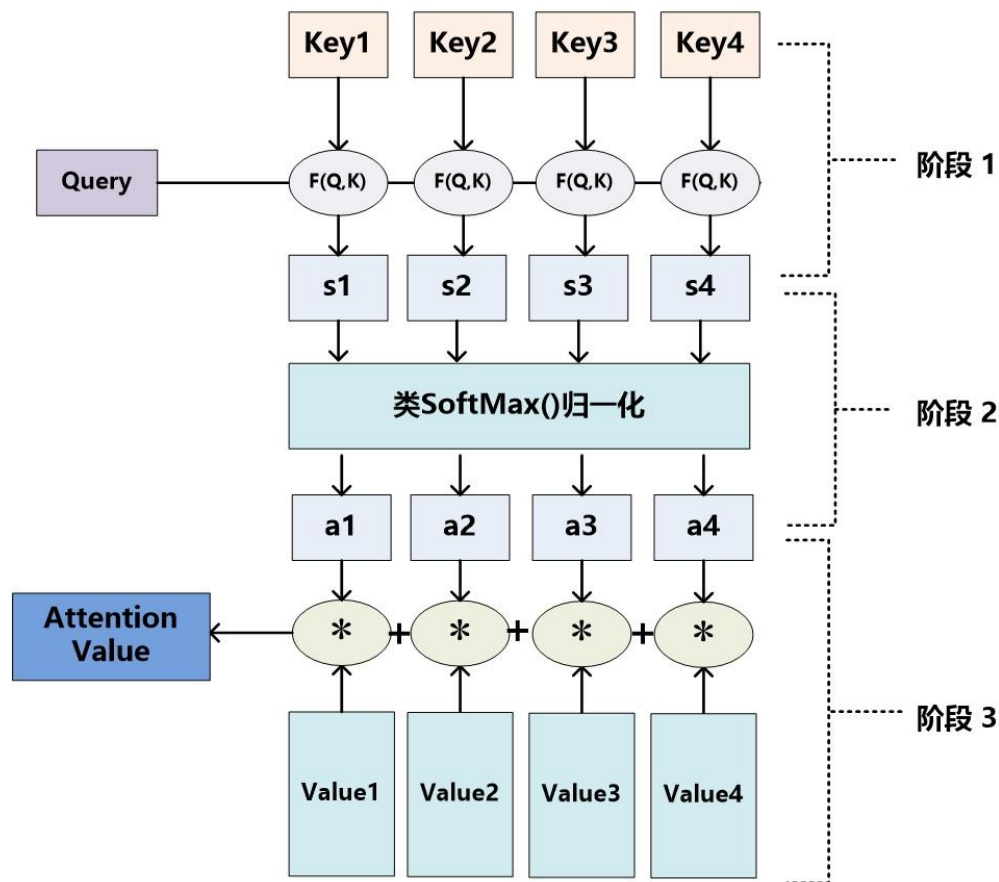


$$\text{Attention}(\text{Query}, \text{Source}) = \sum_{i=1}^{L_x} \text{Similarity}(\text{Query}, \text{Key}_i) * \text{Value}_i$$

# Seq2Seq Attention计算过程(另一种理解方式)



# Seq2Seq Attention计算过程(另一种理解方式)

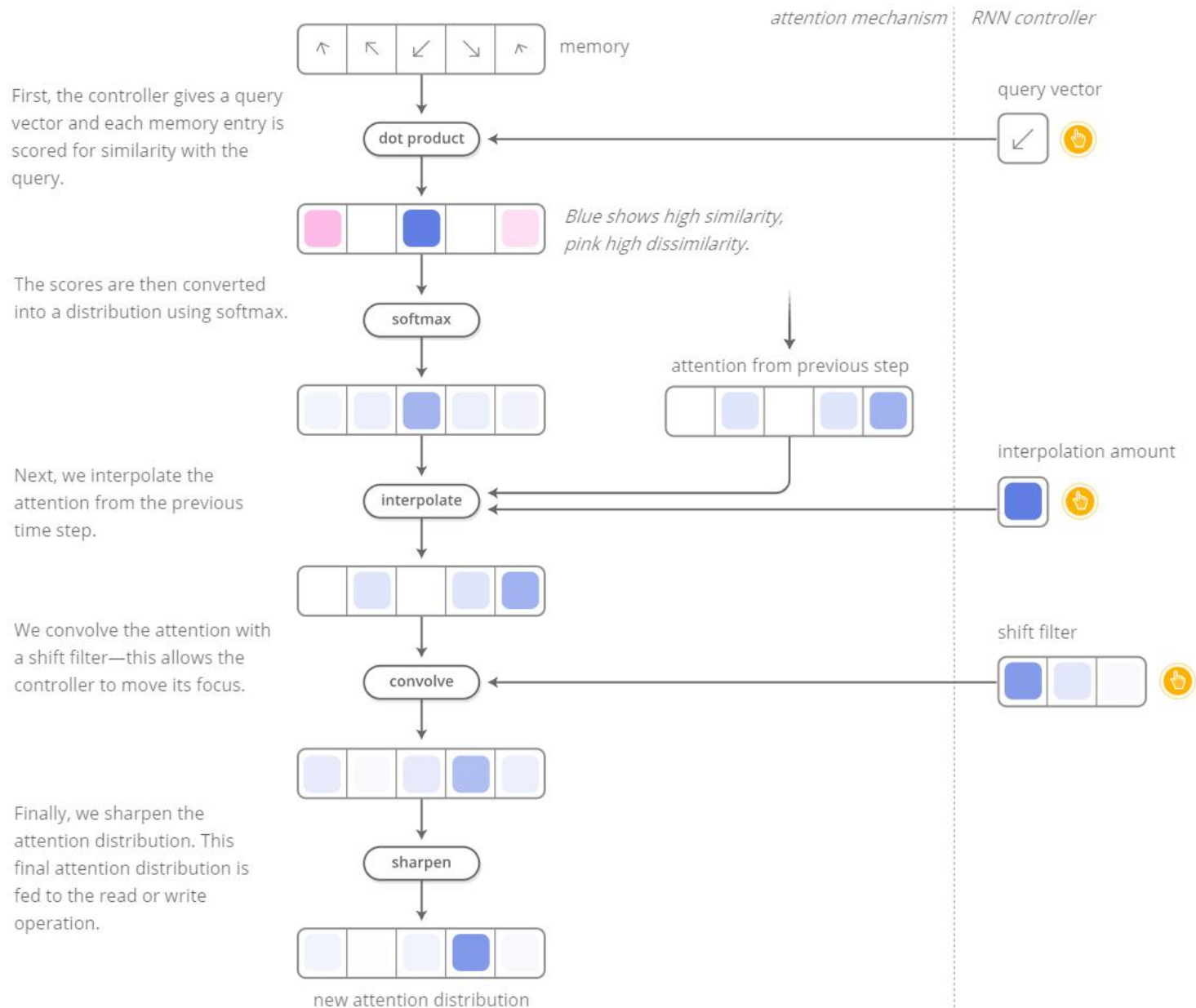


点积:  $\text{Similarity}(\text{Query}, \text{Key}_i) = \text{Query} \cdot \text{Key}_i$

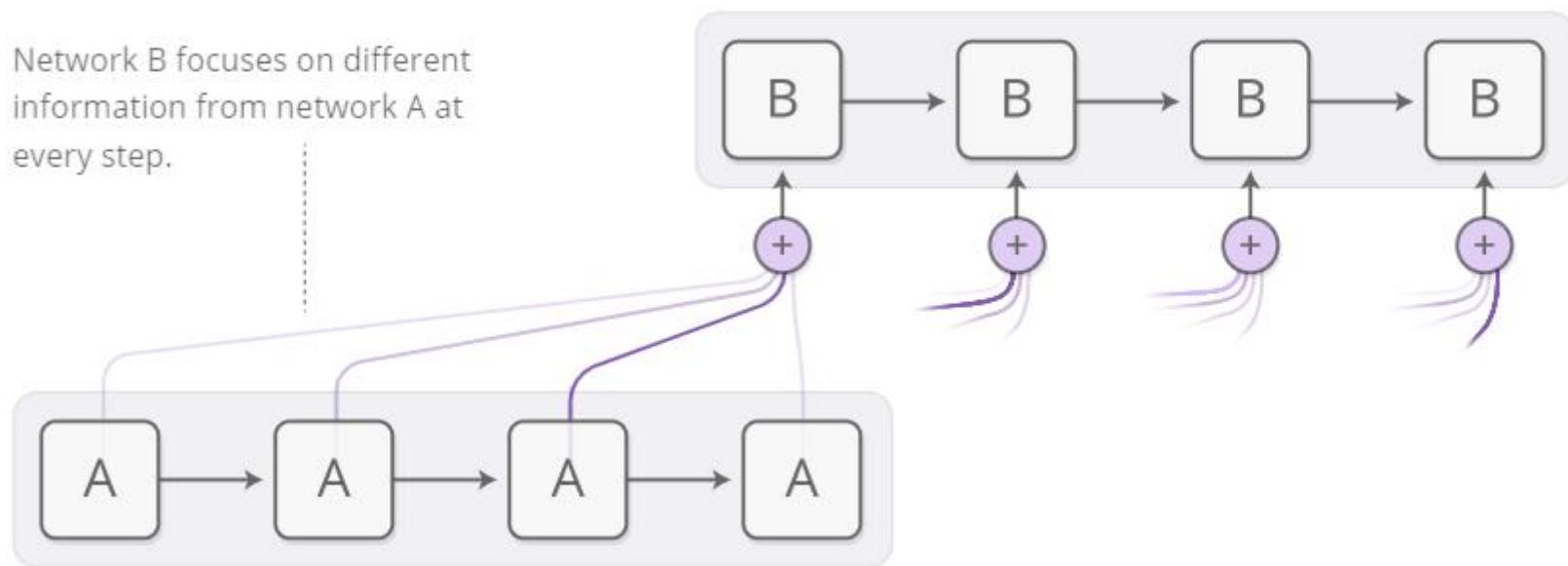
Cosine 相似性:  $\text{Similarity}(\text{Query}, \text{Key}_i) = \frac{\text{Query} \cdot \text{Key}_i}{\|\text{Query}\| \cdot \|\text{Key}_i\|}$

MLP 网络:  $\text{Similarity}(\text{Query}, \text{Key}_i) = \text{MLP}(\text{Query}, \text{Key}_i)$

# Seq2Seq Attention计算过程(另另一种理解方式)

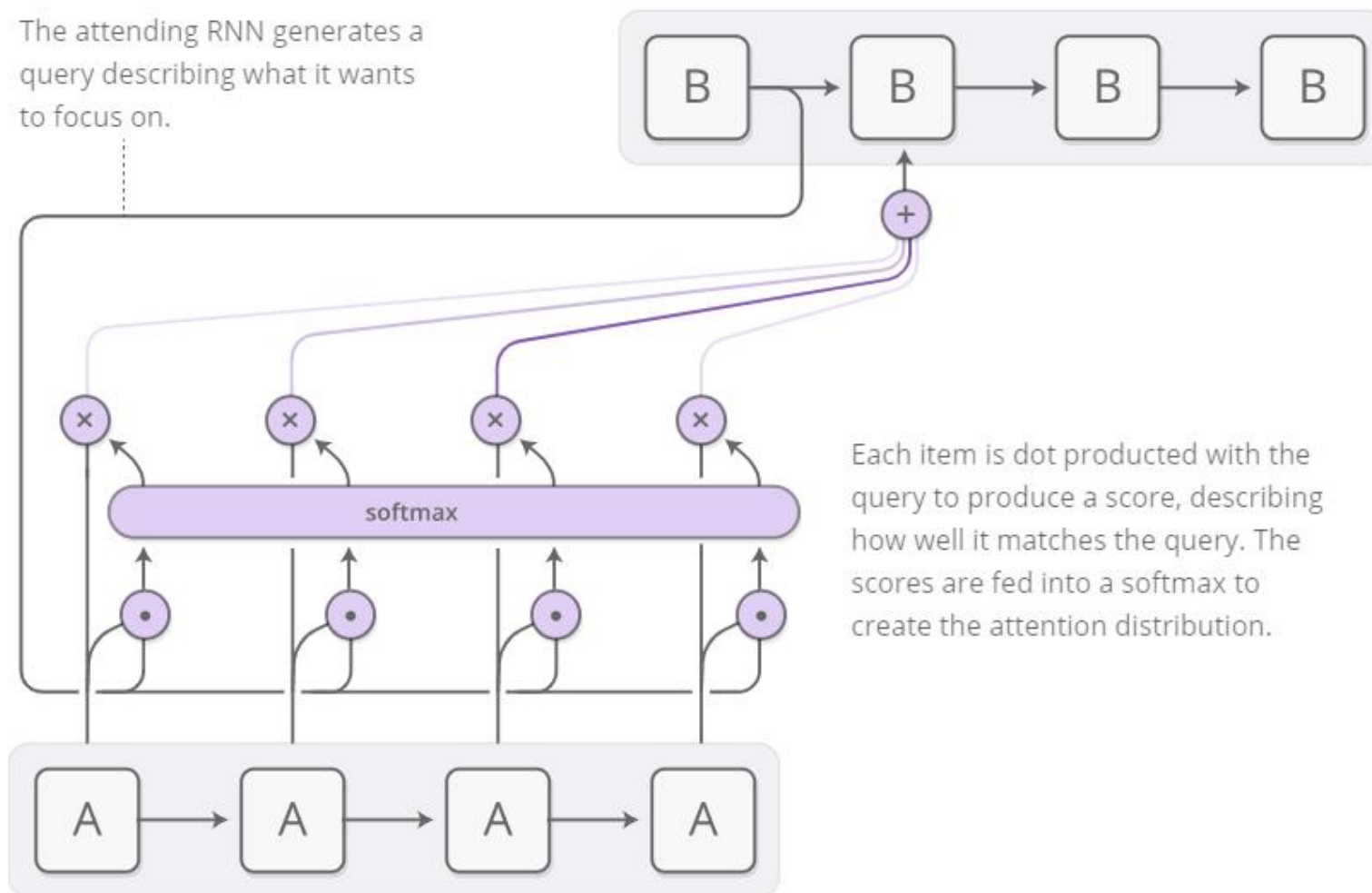


## Seq2Seq Attention计算过程(另另一种理解方式)

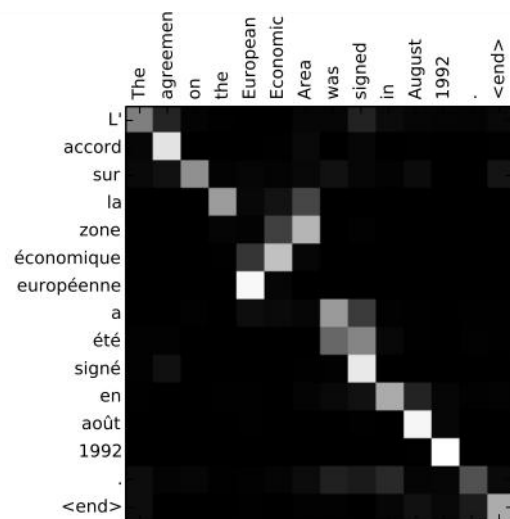




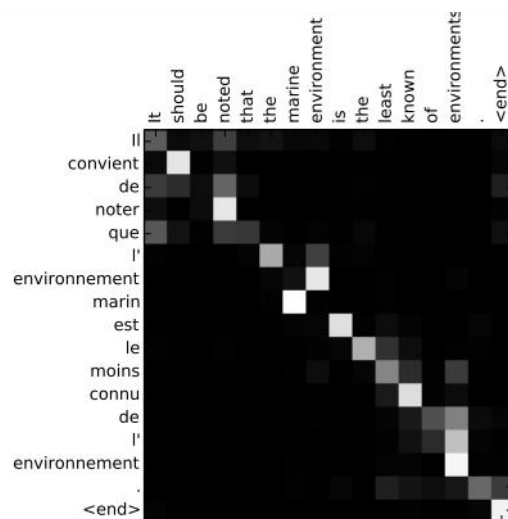
# Seq2Seq Attention计算过程(另另一种理解方式)



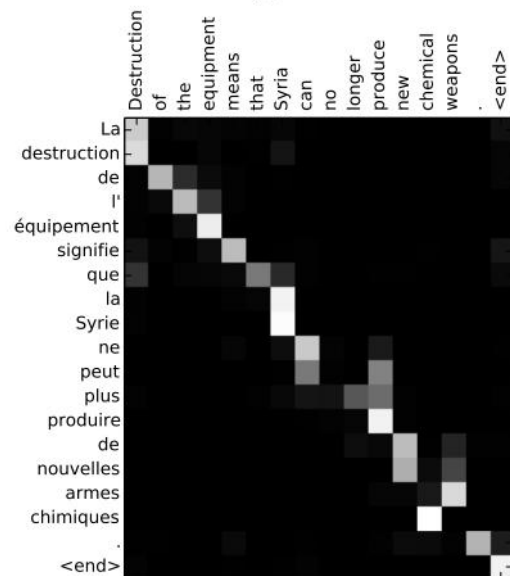
# Seq2Seq Attention效果



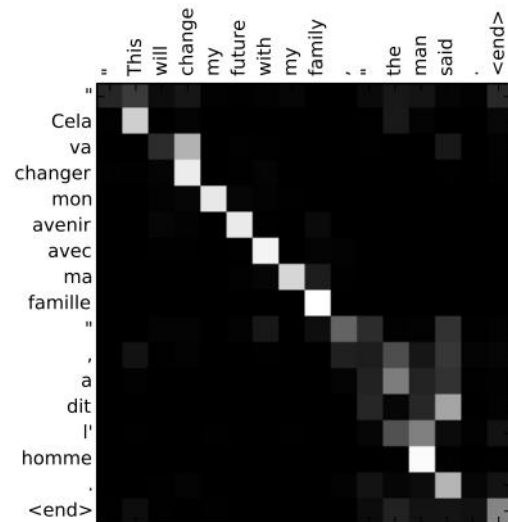
(a)



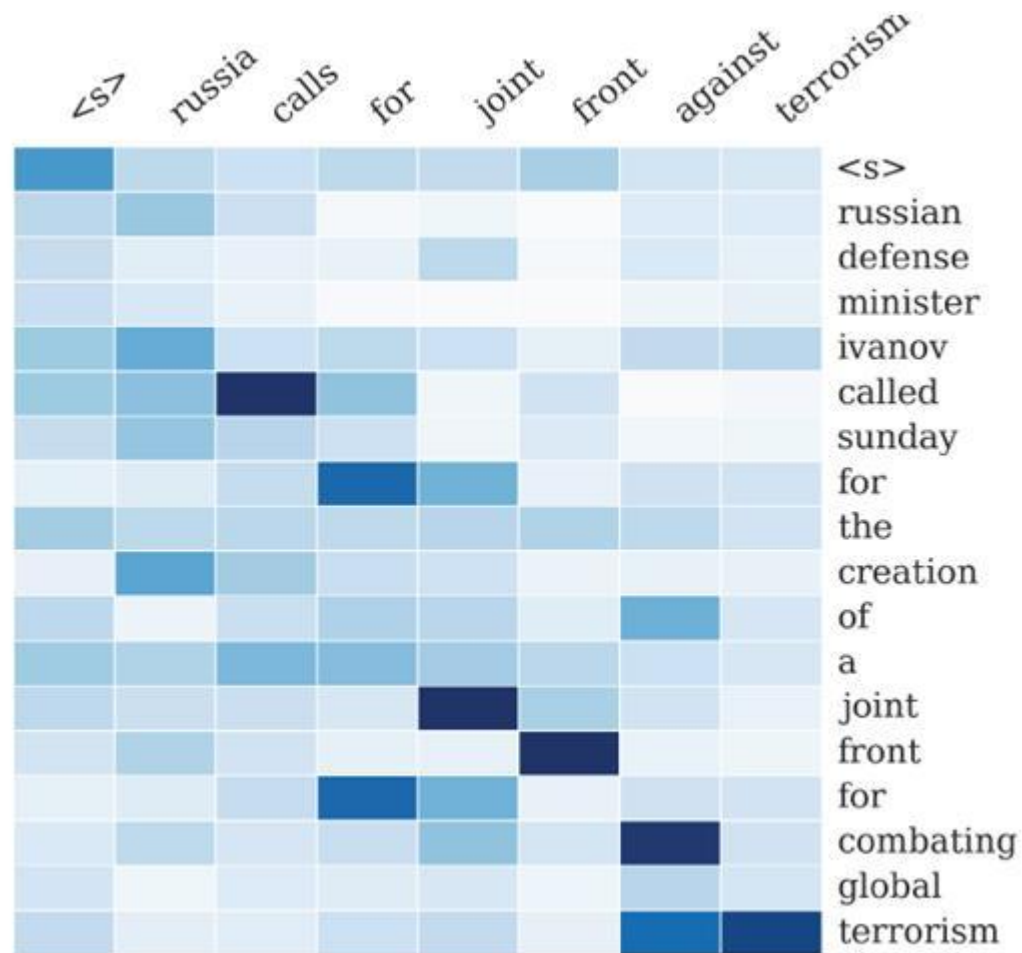
(b)



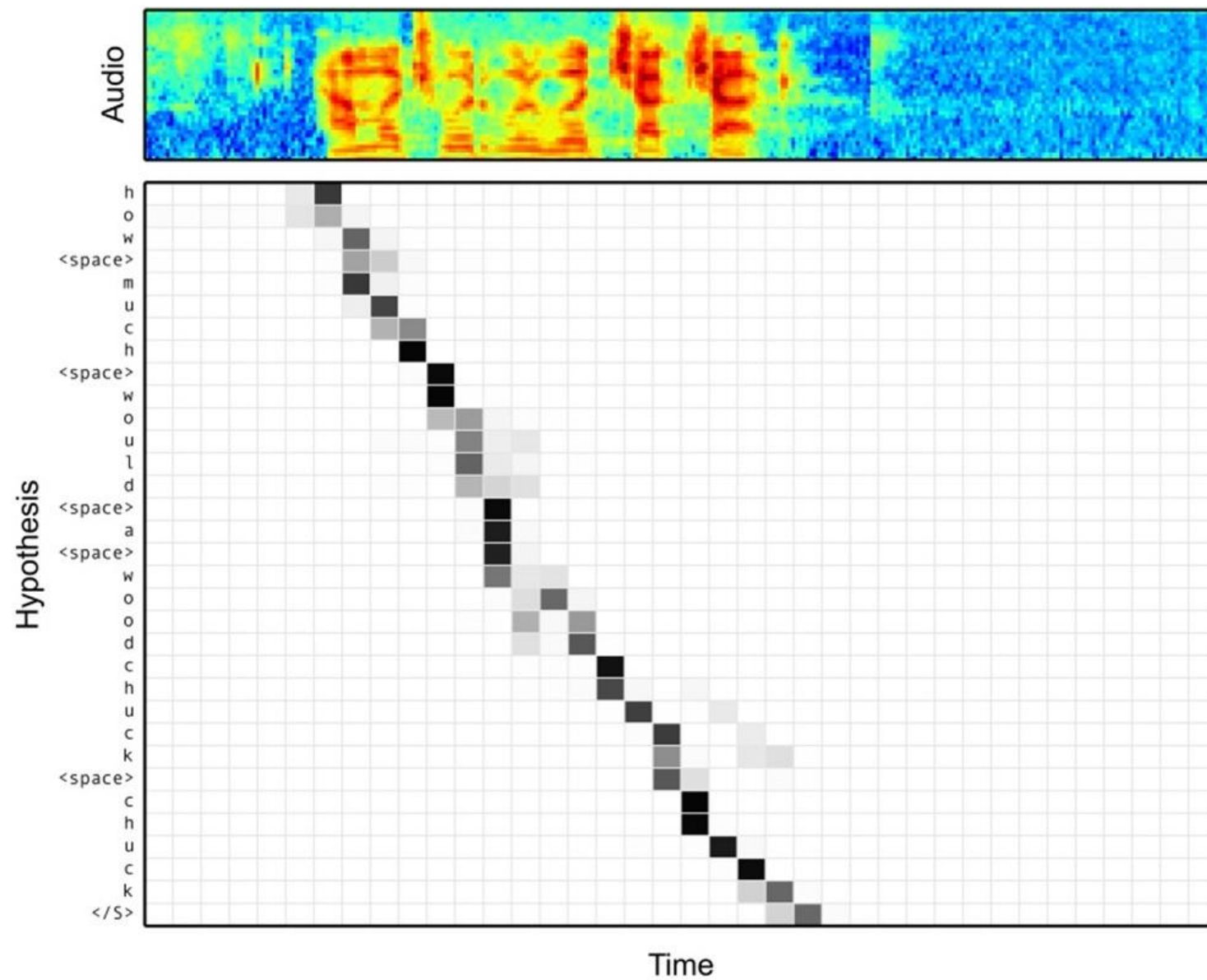
(c)



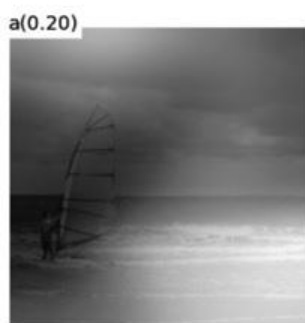
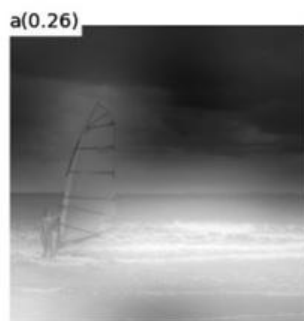
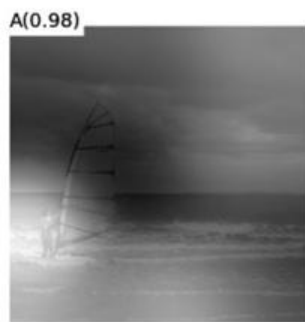
(d)



# Seq2Seq Attention效果



# Seq2Seq Attention效果

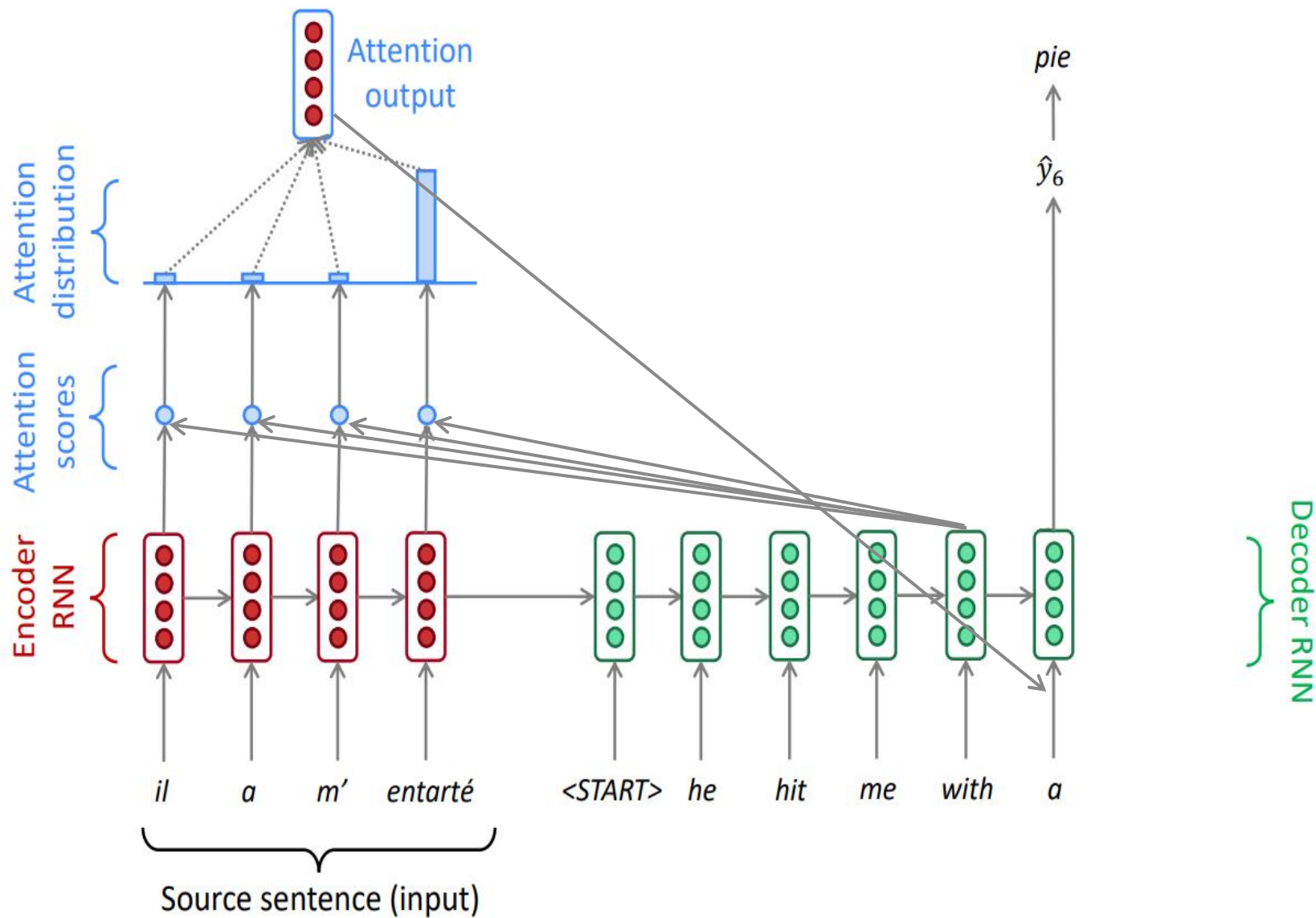


A woman is throwing a frisbee in a park.



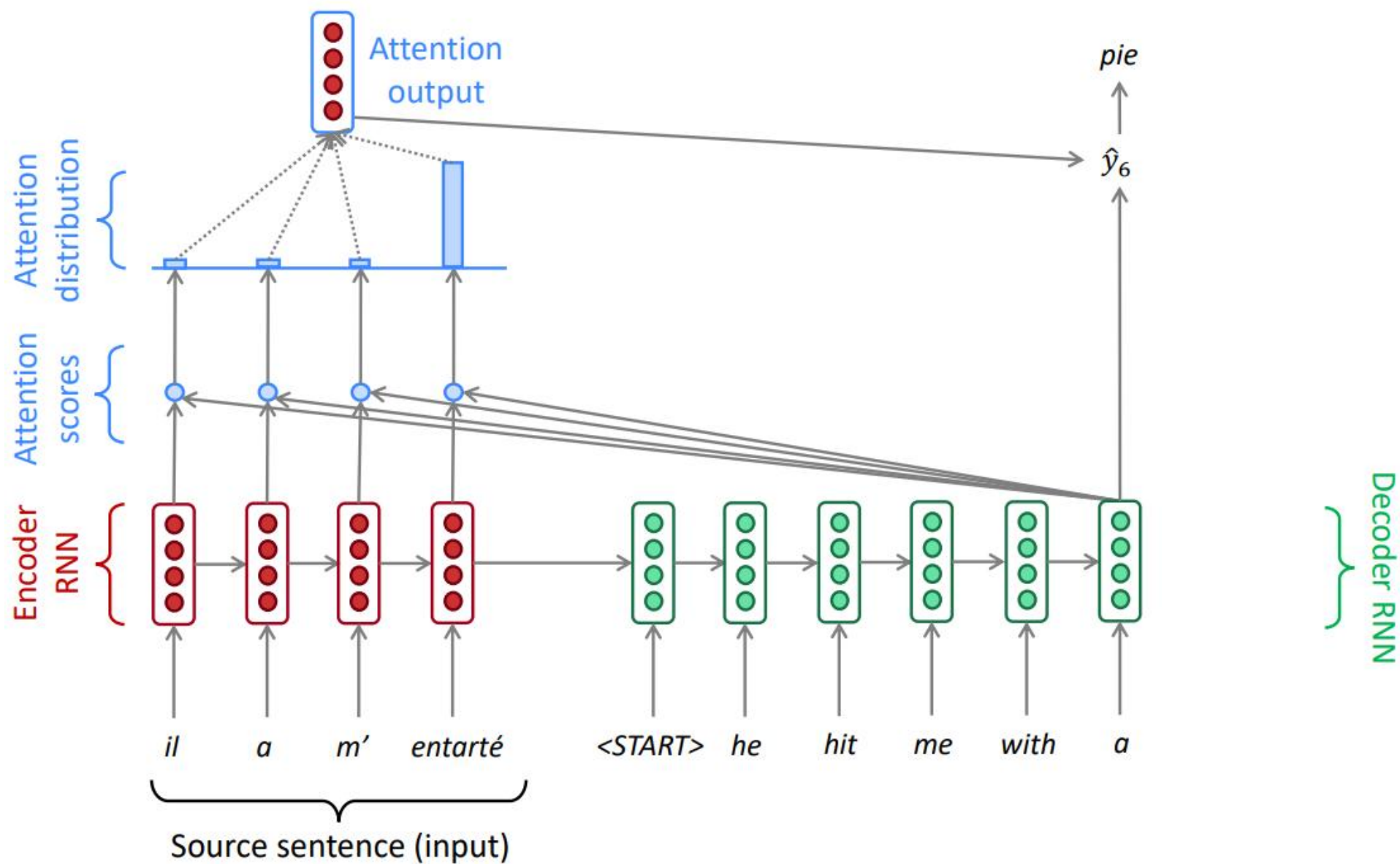
A little girl sitting on a bed with a teddy bear.

# Seq2Seq Attention常规形状一



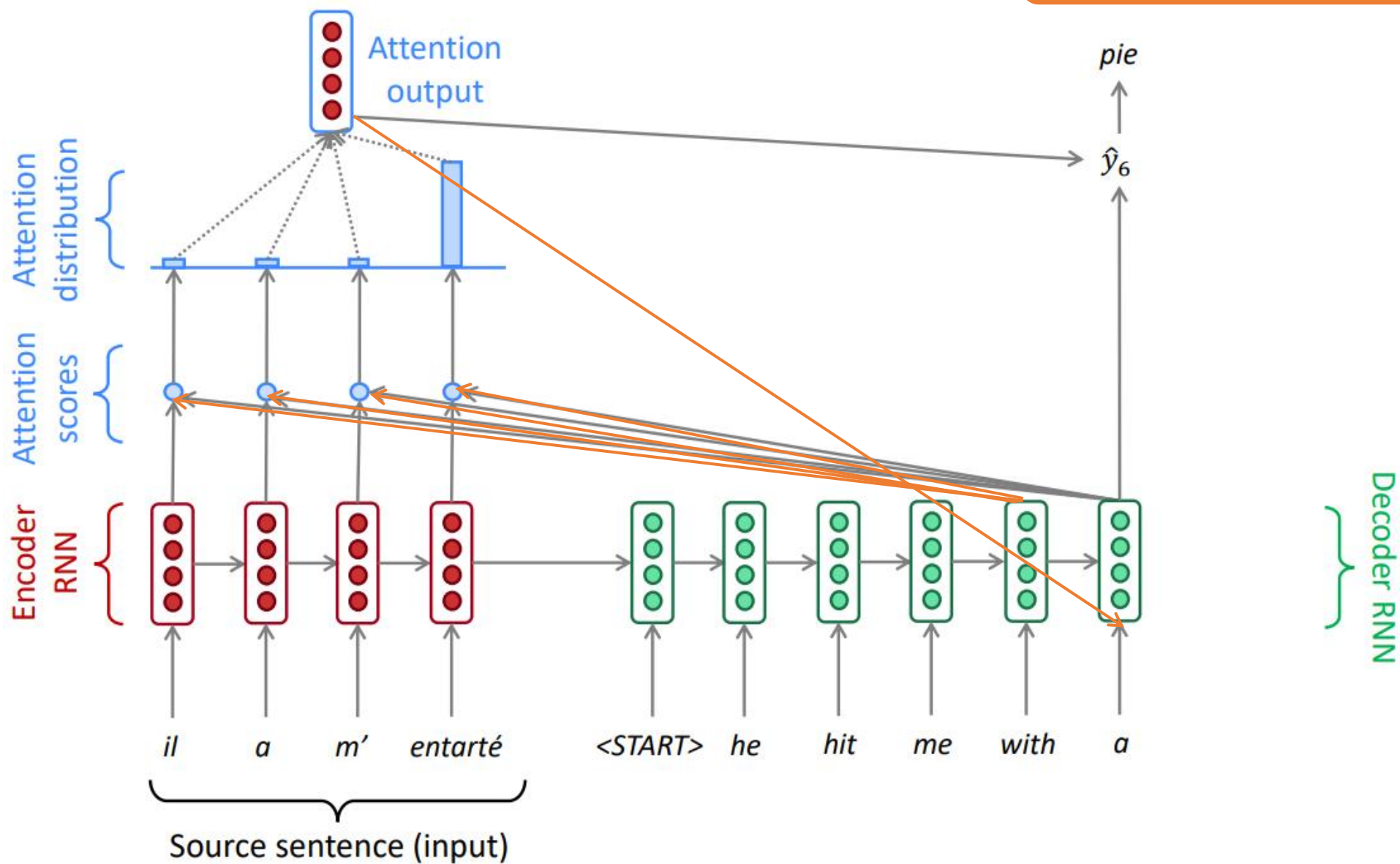


## Seq2Seq Attention常规形状二



# Seq2Seq Attention常规形状三

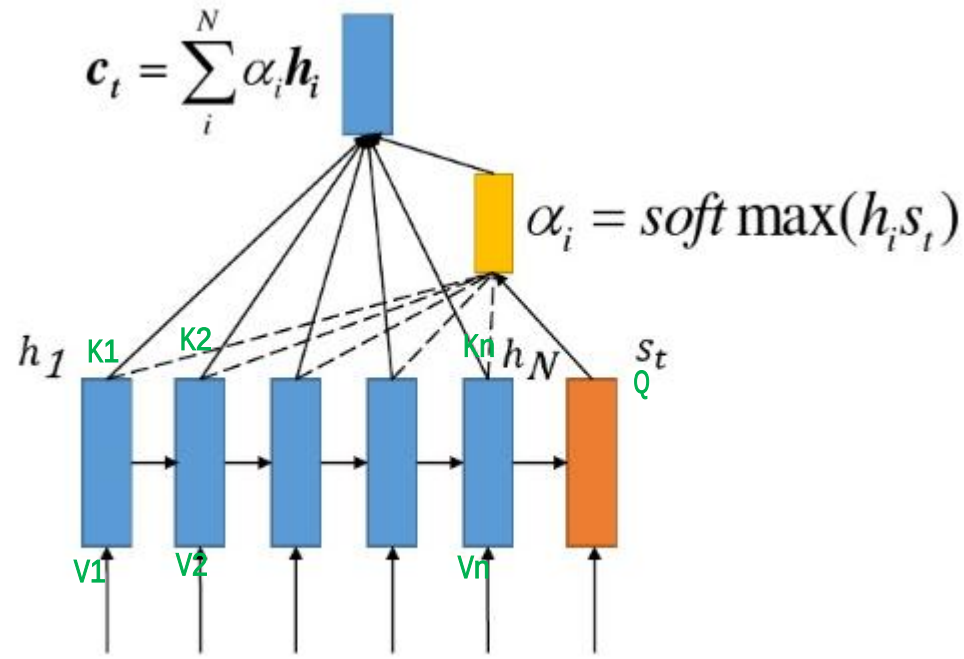
TensorFlow内部实现结构





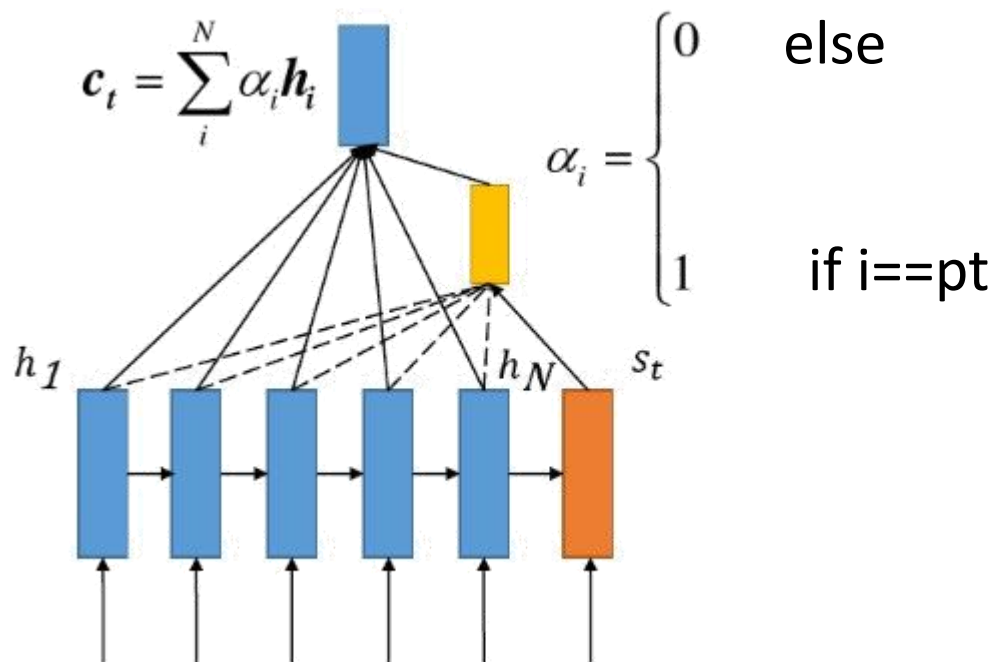
# Seq2Seq Attention Soft Attention

- ⌚ 15年被提出于《Show, Attend and Tell: Neural Image Caption Generation with Visual Attention, Kelvin Xu》



# Seq2Seq Attention Hard Attention

- ⌚ 和Soft Attention在15年同时在一篇论文中被提出;
- ⌚ Soft Attention中是对于每个Encoder的Hidden State会match一个概率值, 而在Hard Attention会直接找一个特定的单词概率为1, 而其它对应概率为0.



# Seq2Seq Attention Global Attention

🕒 在15年被提出于《Effective Approaches to Attention-based Neural Machine Translation, Minh-Thang Luong》, 和Soft Attention类似。

$$h_j = f(h_{j-1}, s)$$

$$\tilde{h}_t = \tanh(W_c[c_t; h_t])$$

$$p(y_t|y_{<t}, x) = \text{softmax}(W_s \tilde{h}_t)$$

$$a_t(s) = \text{align}(h_t, \bar{h}_s) = \frac{\exp(\text{score}(h_t, \bar{h}_s))}{\sum_{s'} \exp(\text{score}(h_t, \bar{h}_{s'}))}$$
$$\text{score}(h_t, \bar{h}_s) = \begin{cases} h_t^\top \bar{h}_s & \text{dot} \\ h_t^\top W_a \bar{h}_s & \text{general} \\ v_a^\top \tanh(W_a[h_t; \bar{h}_s]) & \text{concat} \end{cases}$$
$$a_t = \text{softmax}(W_a h_t) \quad \text{location}$$

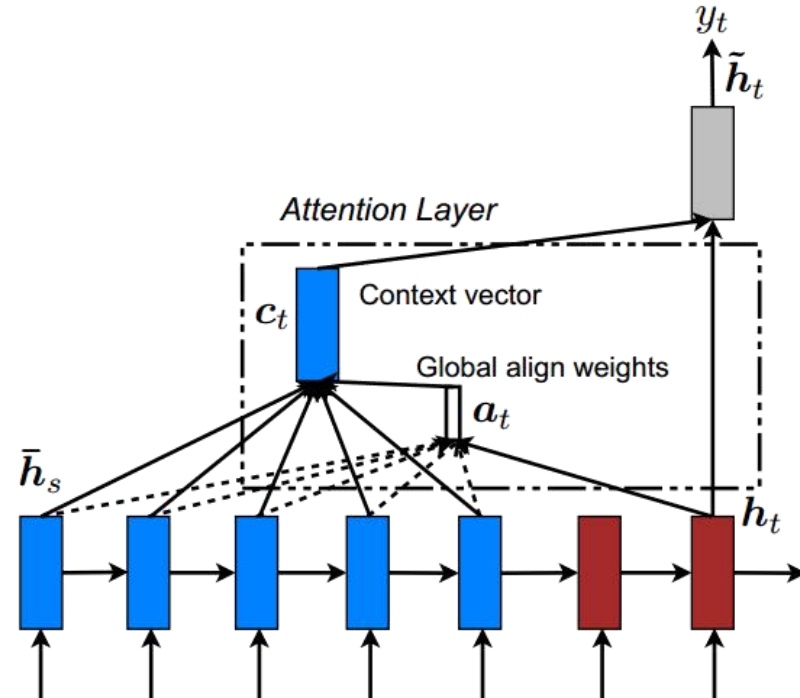
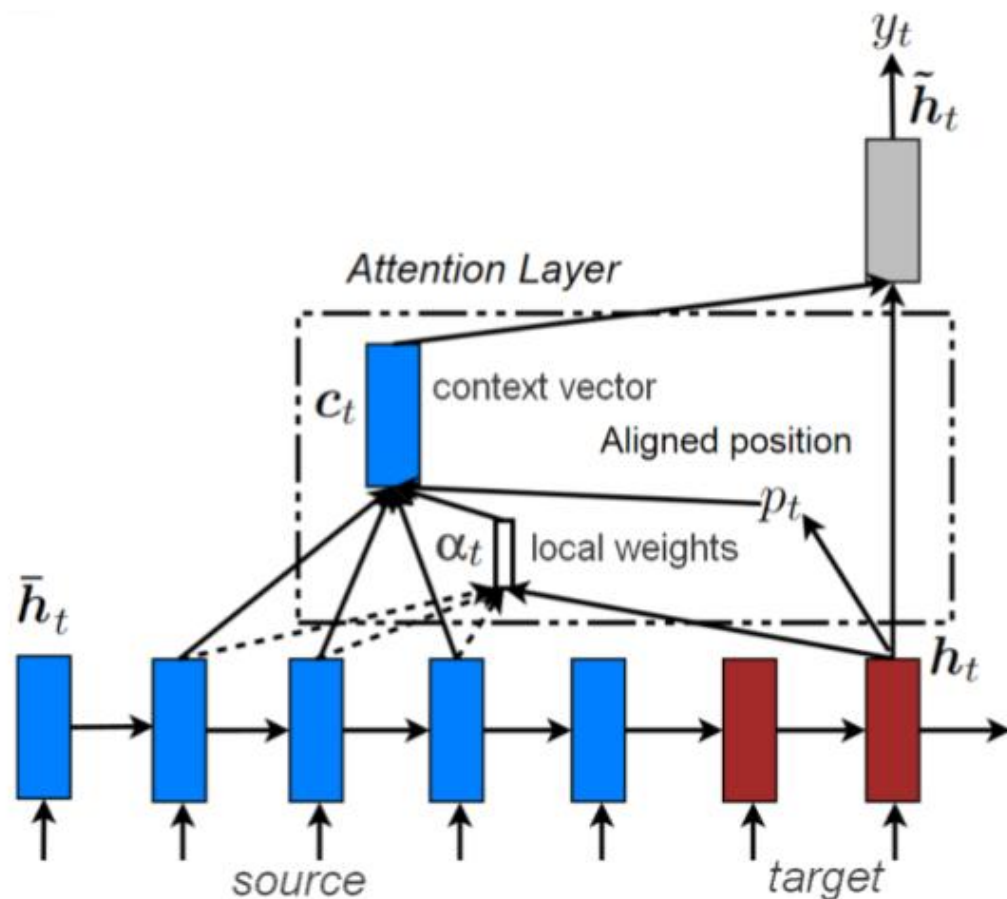


Figure 2: **Global attentional model** – at each time step  $t$ , the model infers a *variable-length* alignment weight vector  $a_t$  based on the current target state  $h_t$  and all source states  $\bar{h}_s$ . A global context vector  $c_t$  is then computed as the weighted average, according to  $a_t$ , over all the source states.

## Seq2Seq Attention Local Attention

- ⌚ 和Global Attention在同一篇论文中被提出；相当于Soft Attention和Hard Attention中间状态(半硬半软Attention)
- ⌚ 对于时刻 $t$ 的词汇，模型首先产生一个对齐位置 $p_t$ (aligned position), context vector( $c$ )由编码器中的隐状态计算得到，编码器的隐状态不是所有的隐状态，而是在区间 $[p_t-D, p_t+D]$ 中， $D$ 的大小由经验给定。

# Seq2Seq Attention Local Attention



$$P[y_t | \{y_1, \dots, y_{t-1}\}, c_t] = \text{softmax}(W_s \tilde{h}_t)$$

attentional hidden state

$$\tilde{h}_t = \tanh(W_c [c_t; h_t])$$

decoder hidden state

context vector

$$p_t = T_x \cdot \sigma(v_p^\top \tanh(W_p h_t))$$

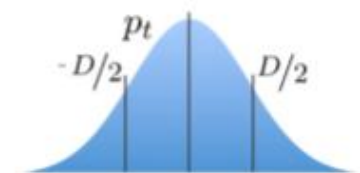
$i^{\text{th}}$  encoder hidden state

$$c_t = \sum_{i=p_t-D}^{p_t+D} \alpha_{t,i} \bar{h}_i$$

alignment vector

$$\alpha_{t,i} = \frac{\exp(\text{score}(h_t, \bar{h}_i))}{\sum_{i'=p_t-D}^{p_t+D} \exp(\text{score}(h_t, \bar{h}_{i'}))} \exp\left(-\frac{(i-p_t)^2}{2(D/2)^2}\right)$$

$\text{score}(h_t, \bar{h}_i) = h_t^\top W_\alpha \bar{h}_i$



# Seq2Seq Attention Self Attention

⌚ 在17年被提出于《Attention Is All You Need, Ashish Vaswani》，也称为Transformer结构；内部包含Multi-Head Attention以及Residual结构。

⌚ Transformer是Bert、LLM等网络结构的基础。

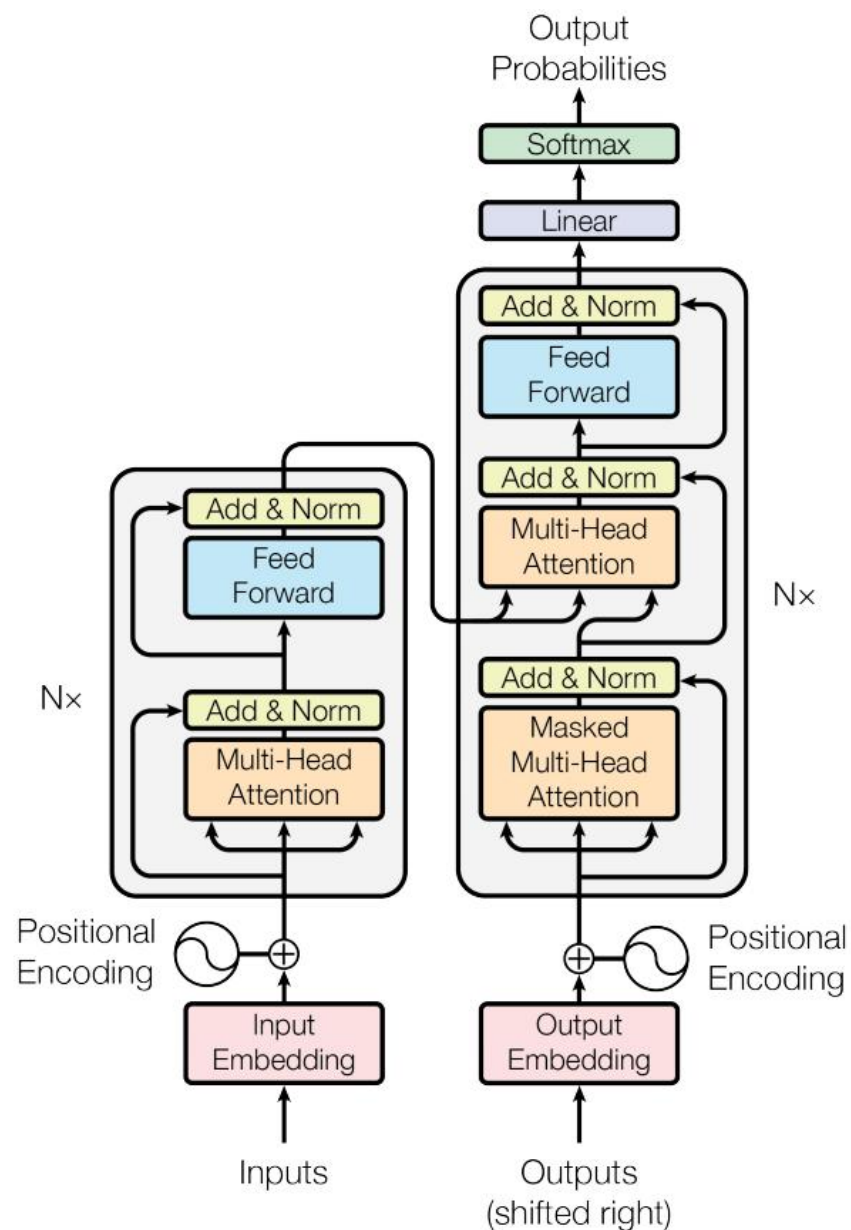
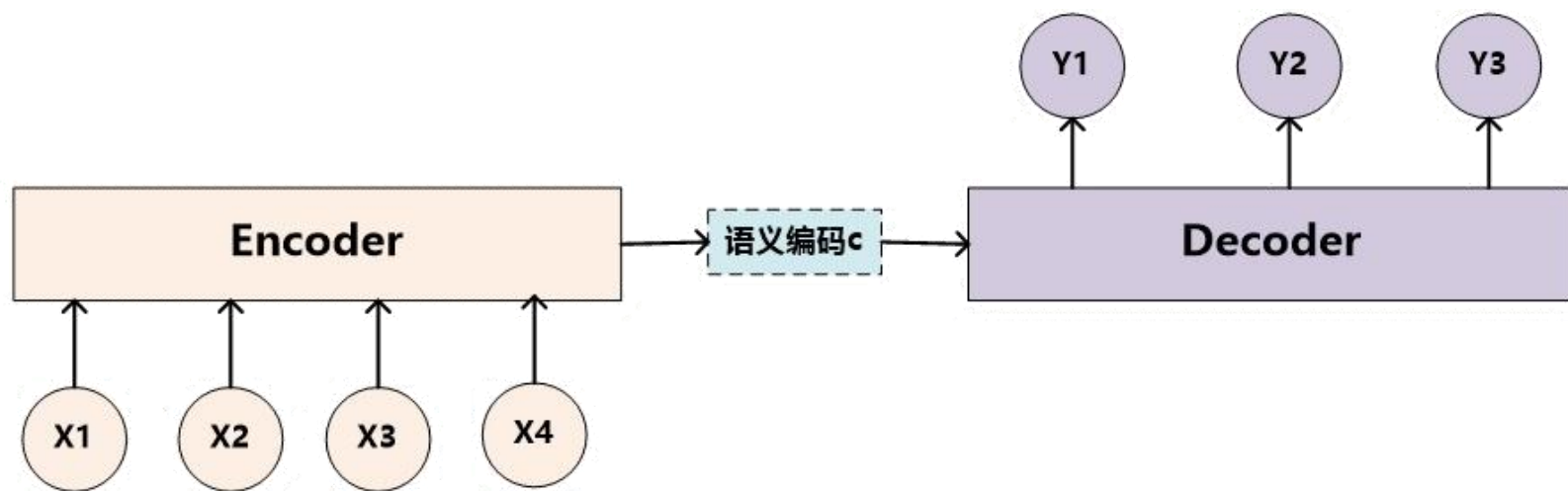


Figure 1: The Transformer - model architecture.



# 总结\_Seq2Seq\_Attention

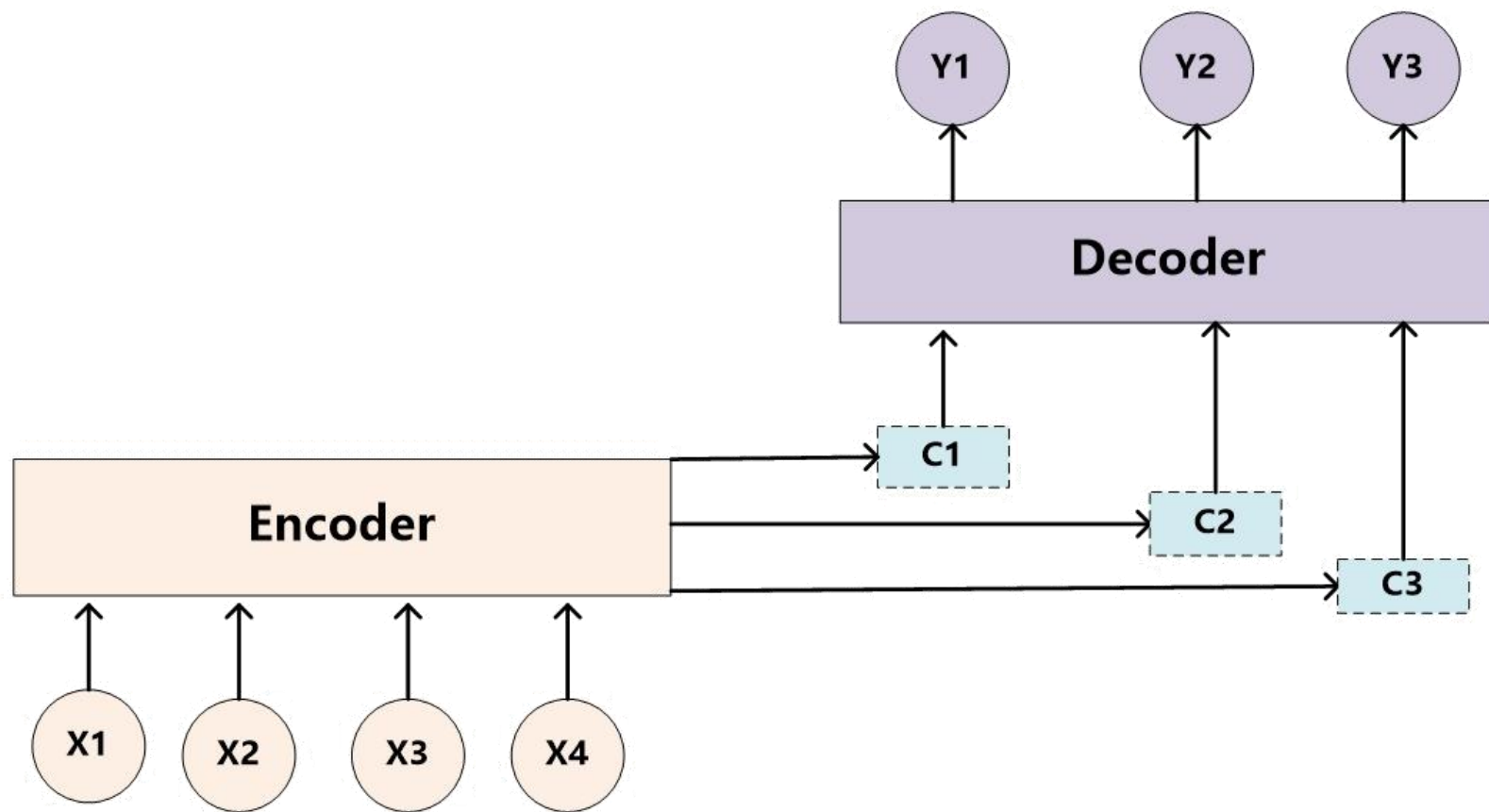


$$Y_1 = f(C)$$

$$Y_2 = f(C, Y_1)$$

$$Y_3 = f(C, Y_1, Y_2)$$

## 总结\_Seq2Seq\_Attention

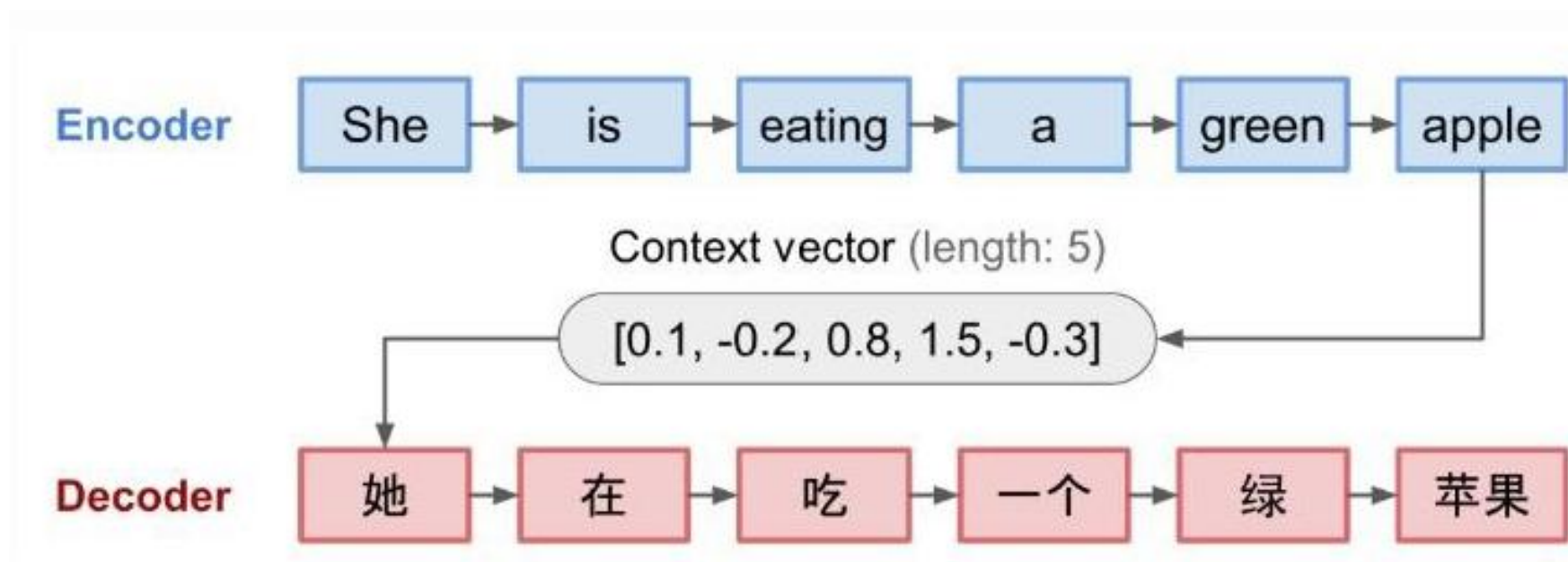


$$Y_1 = f(C_1)$$

$$Y_2 = f(C_2, Y_1)$$

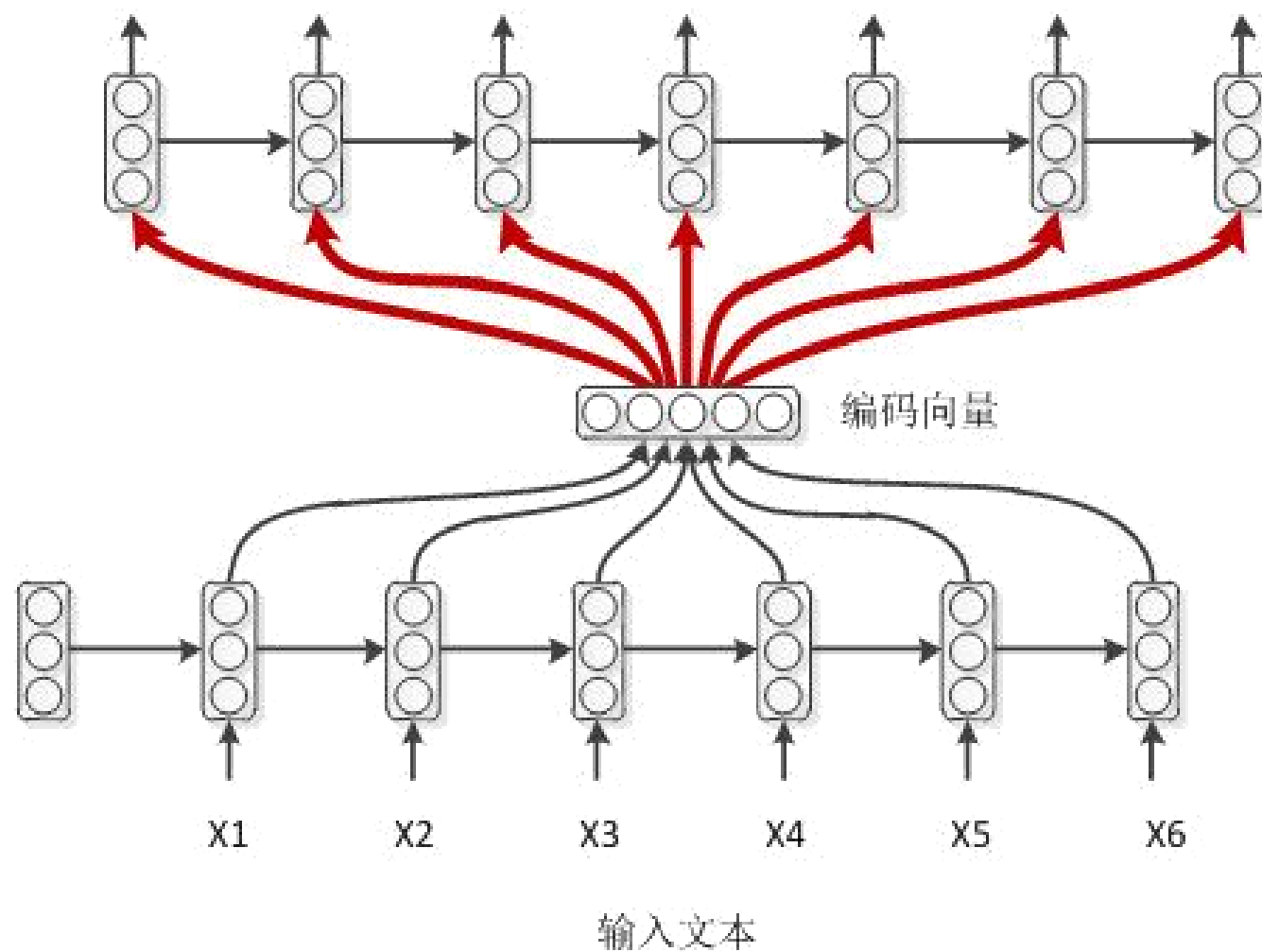
$$Y_3 = f(C_3, Y_1, Y_2)$$

# 总结\_Seq2Seq\_Attention



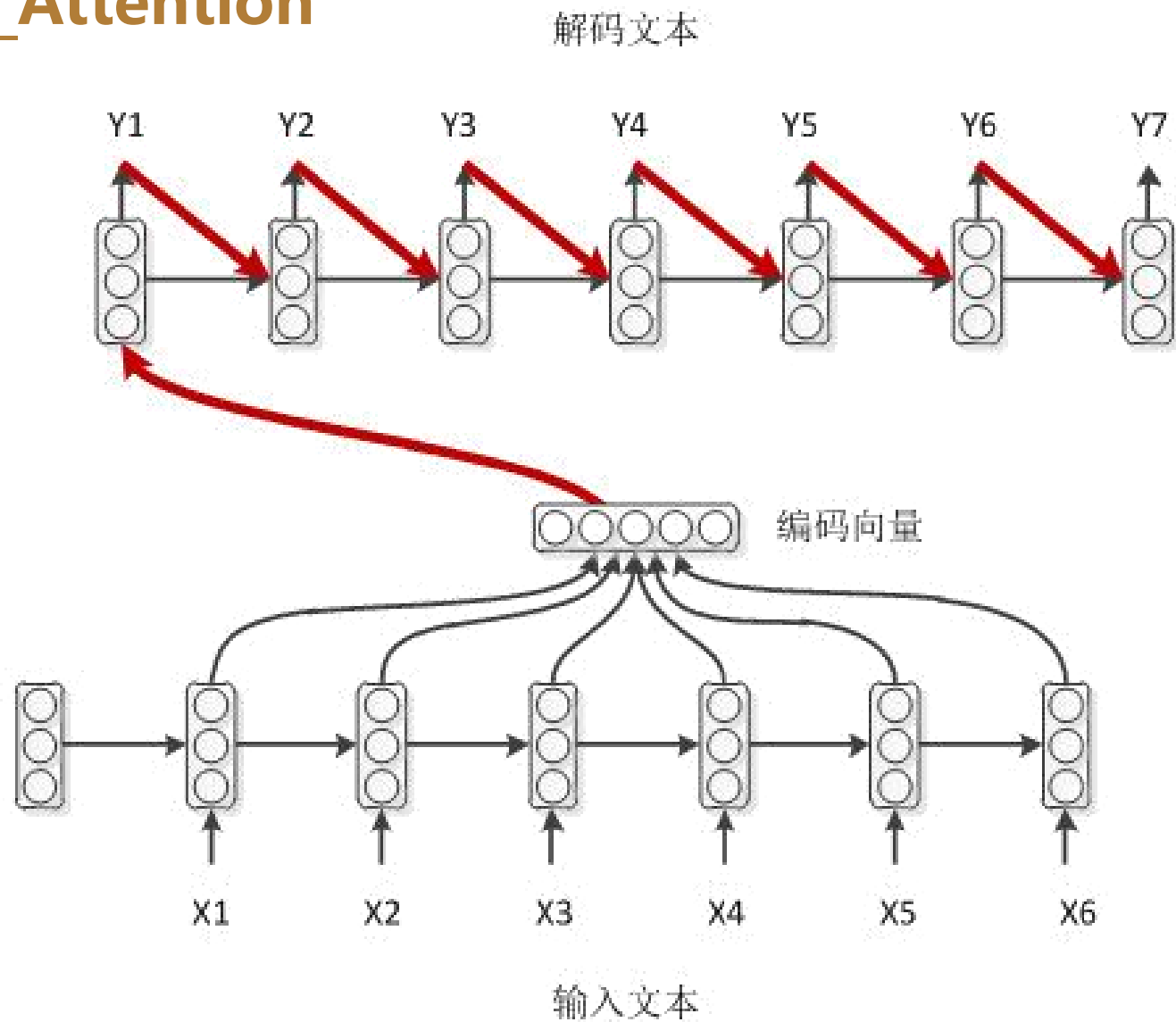
# 总结\_Seq2Seq\_Attention

解码文本



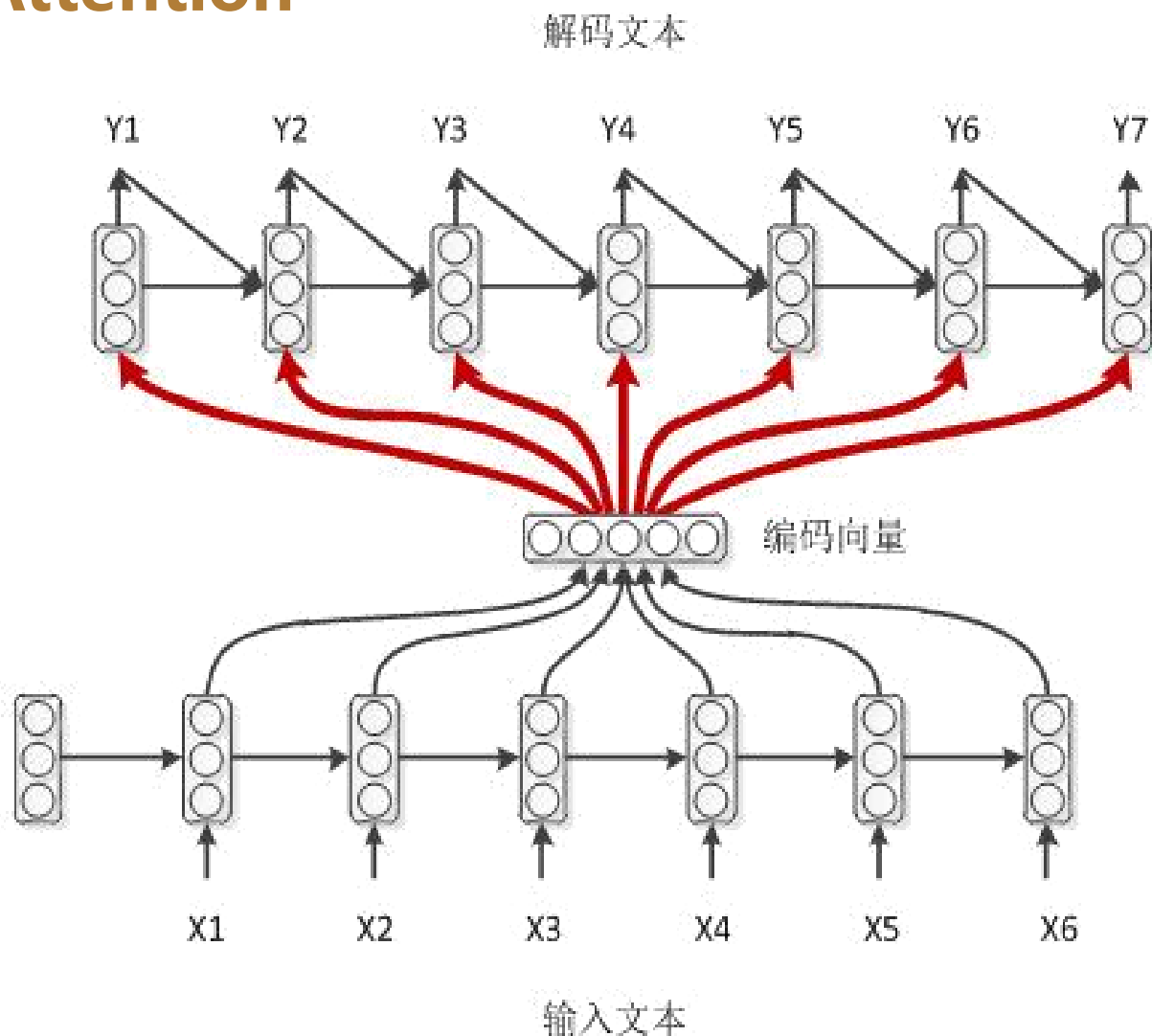
- 最简单的解码模式 – Decoder 1

# 总结\_Seq2Seq\_Attention



- 带输出回馈的解码模式 – Decoder 2

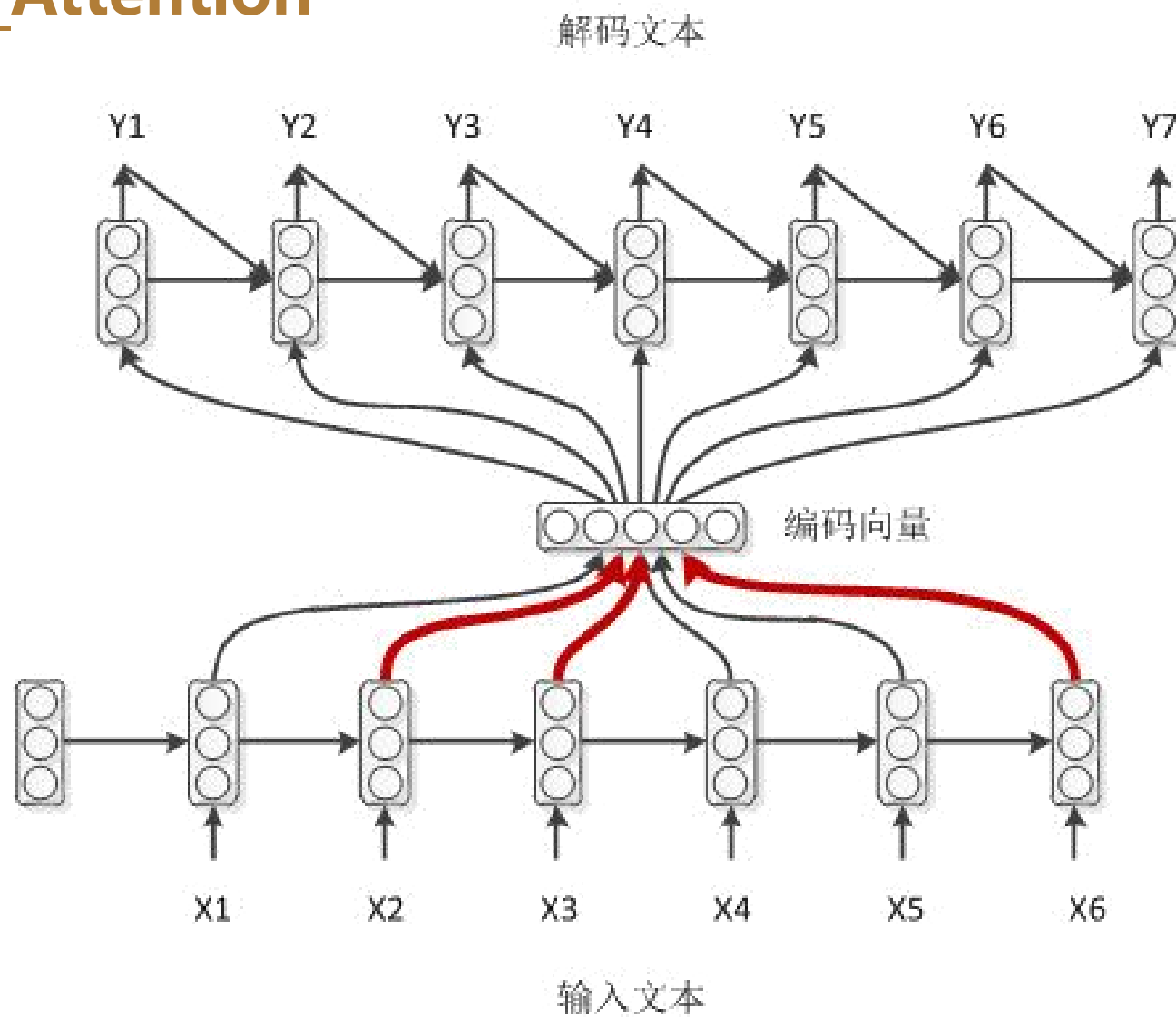
# 总结\_Seq2Seq\_Attention



- 带编码向量的解码模式– Decoder 3

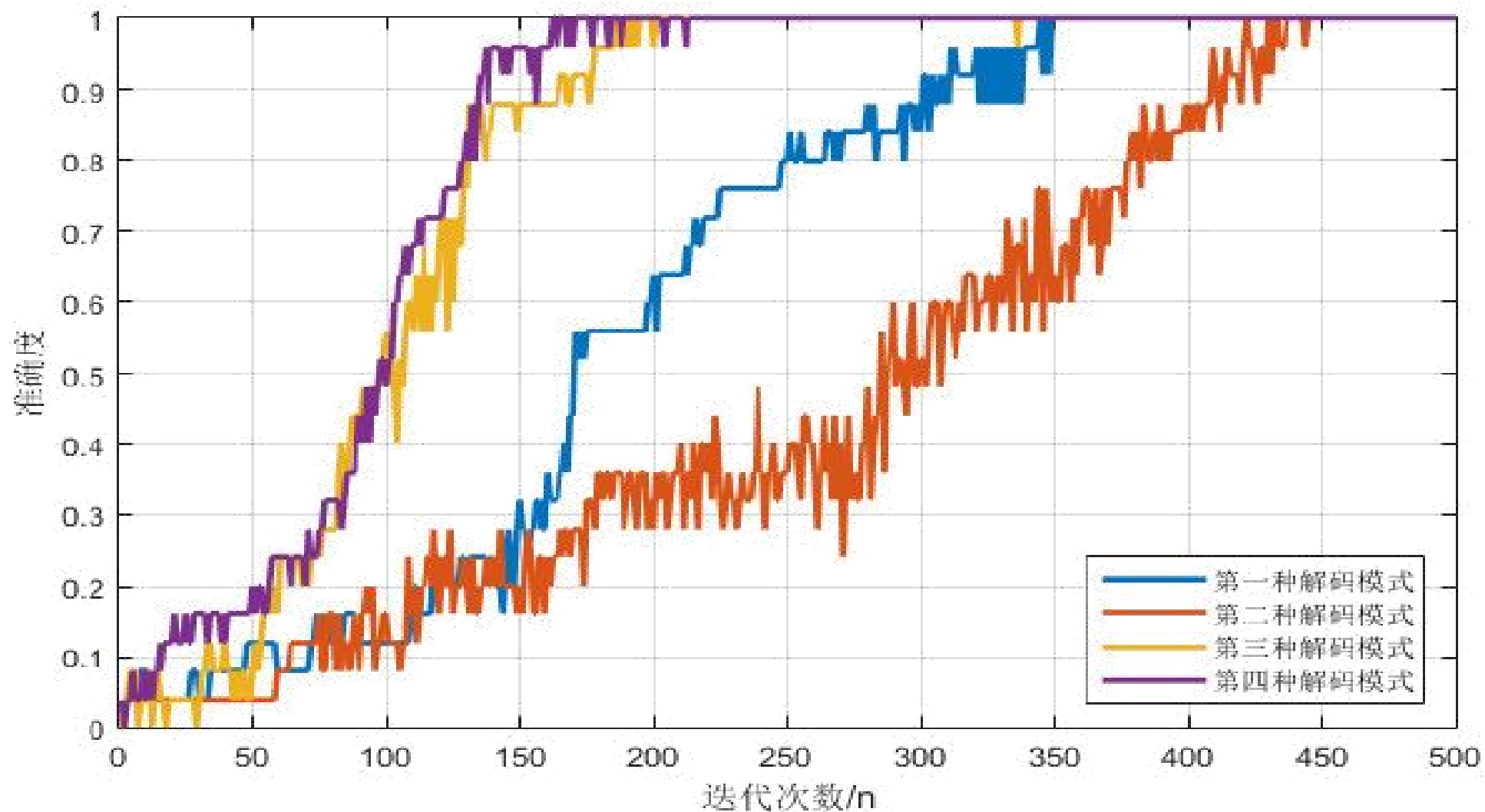


# 总结\_Seq2Seq\_Attention

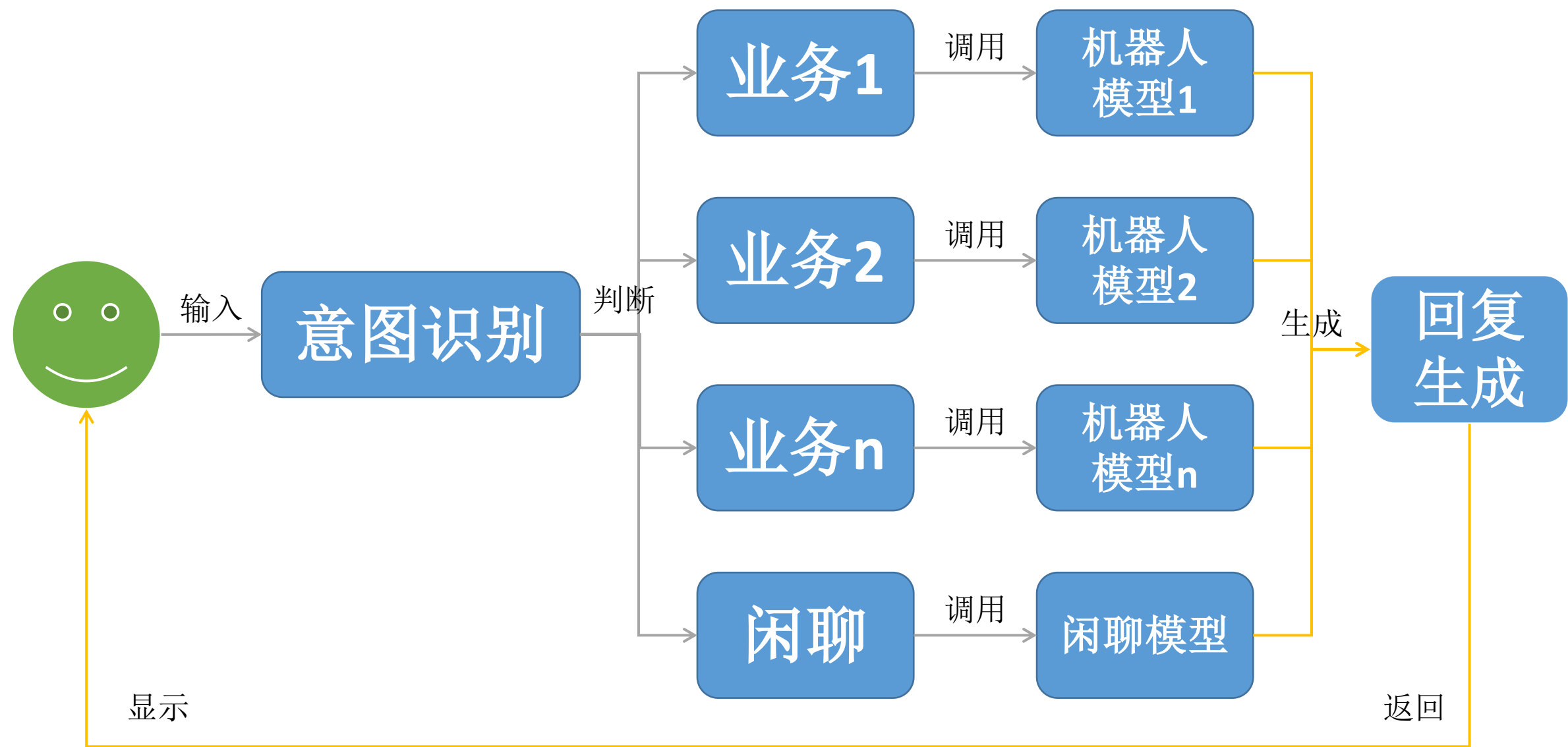


- 带注意力的解码模式- Decoder 4

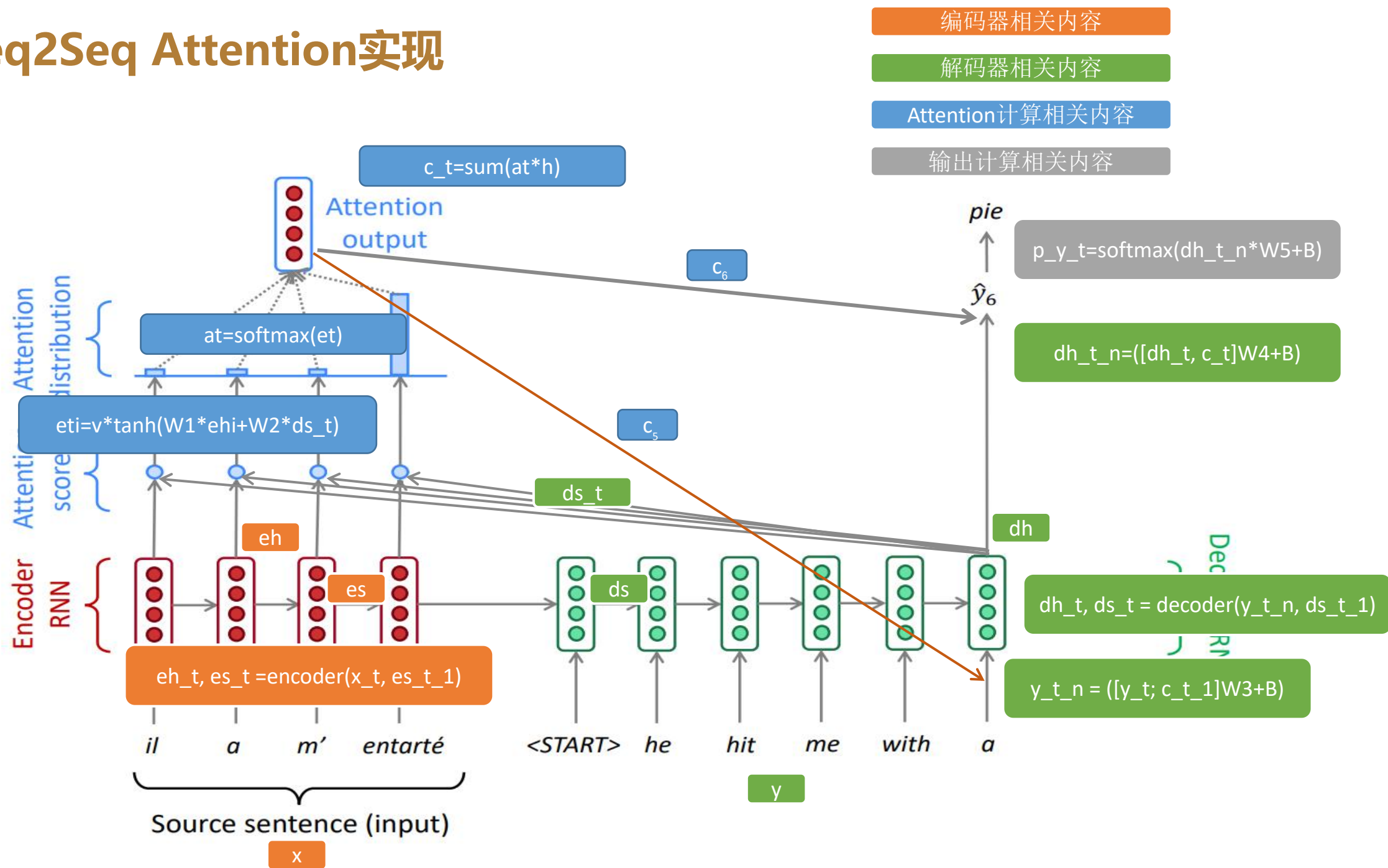
## 总结\_Seq2Seq\_Attention



# Seq2Seq+Attention项目\_聊天机器人



# Seq2Seq Attention实现



THANKS!